

PrimeSim™ MOSRA User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	7
Related Products, Publications, and Trademarks	7
Conventions	7
Customer Support	8
Statement on Inclusivity and Diversity	9

1. Reliability Analysis	10
Introduction to the PrimeSim MOSRA Tool	10
PrimeSim MOSRA Simulation Flow	11
MOSRA Supported Models	11
MOSRA Simulation Setup — Examples	13

2. PrimeSim MOSRA Reliability Models	15
BTI and HCI Models	15
Level 1 BTI and HCI Model	16
Level 3 BTI and HCI Model	16
BTI Model	16
Synopsys LEVEL 3 BTI Model Parameters	17
HCI Model	18
Synopsys LEVEL 3 HCI Model Parameters	18
Custom Reliability Model Using MOSRA API	19

3. PrimeSim MOSRA Simulation Commands	21
.MOSRA	21
.MOSRAPRINT	30
.MOSRA_SUBCKT_PIN_VOLT	31
.MODEL	32
.APPENDMODEL	33

4. MOSRA Control Options Reference	37
---	-----------

.OPTION APPENDALL	38
.OPTION DEGF	39
.OPTION DEGFN	39
.OPTION DEGFP	40
.OPTION DEGFM	40
.OPTION MRA_POST_COMPUTE	41
.OPTION MOSRALIFE	41
.OPTION MOSRAPATH	41
.OPTION MOSRASORT	42
.OPTION MOSRASORTMODE	43
.OPTION MRAAPI	43
.OPTION MRADTEMPBA	44
.OPTION MRAEXT	44
.OPTION MRAMCFILE	45
.OPTION MRAPAGED	45
.OPTION MRA_CONTROL	45
.OPTION MRASHETEMP	47
.OPTION MRA00PATH, MRA01PATH, MRA02PATH, MRA03PATH	48
.OPTION RADEGFILE	48
.OPTION RADEGOUTPUT	49

5. MOSRA Output Files	51
MOSRA Output Files	51
.radeg File	51
.radeg*.csv File	52
Output Files of Post Stress Simulation	52
.#@ra.grp File	53

6. PrimeSim MOSRA Integration in Simulation Environments	54
---	-----------

7. Using PrimeSim MOSRA for Aging Analysis	55
Basic Analysis	55
Stress-Only Simulation	55

Related Commands and Options	56
Example	56
Post Stress Only Simulation	57
Related Commands and Options	57
Example	57
Post Stress Only Simulation at Customized Time	58
Related Commands and Options	58
Example	59
Both Stress and Post Stress Simulation	59
Related Commands and Options	60
Example	60
Gradual Aging Analysis	61
Simmode=2 Based Gradual Aging	61
Related Commands and Options	62
Example	62
Simmode=3 Based Gradual Aging	62
Related Commands and Options	63
Simmode=4 Based Gradual Aging — Custom Aging Accumulation	65
Related Commands and Options	65
Examples	65
Process Variation Aware Aging Analysis	67
Examples	69
Self-Heating Aware Aging Analysis	71
MOSRA for Non-Time Domain Transient SPICE Analyses	72
Related Commands and Options	72
Example	72
8. MOSRA API for Custom Aging Model Development	74
MOSRA API Example Package and MOSRA API Functions and Data Structures	74
Integration-Based Aging Calculation Using MOSRA API	77
Adding Custom Reliability Models	78
Introduction to the Files Under MRA	79
Adding a New MOSRA Model Parameter	81
Defining the MOSRA Model Level and Other Generic Types	82
Custom Aging Formulation Deployment	83
How to Create a New MOSRA Model Directory	85
Creating a New MOSRA Model Directory	85

Contents

Modifying the MOSRAcfg.c File Under the ./access Directory	86
Supported Platforms	87
Compilation Environment Setup	87
Library Compilation	87
MOSRA Library Quality	88
Setting Up the Runtime Shared-Library Path	88
Netlist Setup	89
Other Functions Supported in MOSRA API	89
Accessing Internal Variables of the MOSRA API Models	89
mraPrint() Function	91
MOSRA API Supported Aging Degradations	91
MOSRA Defined Aging Degradations	92
Degradation Types on Base Model Parameters	92
Other Degradation Types	94
BSIM4 Additional Aging Degradation Types	95
BSIM4 Gate-Leakage Aging Degradation Types	96
TFT (Level 62 MOSFET) Model Parameters	100
PSP (Level 69 MOSFET) Model Parameters	100
BSIM-CMG (Level 72 MOSFET) Model Parameters	101
HiSIM-HV (Level 73 MOSFET) Model Parameters	101
UTSOI (Level 76 MOSFET) Model Parameters	102
BSIM-BULK (Level 77 MOSFET) Model Parameters	102
BSIM-IMG (Level 78 MOSFET) Model Parameters	103
IDS Degradation Types for all Models	103
Degradation Types for Instance Parameters	104
Other Degradation Types	104
User-defined Degradation Types	104
MOSRA API Data Structures	105
MOSRA API Model Function Prototypes	106

About the PrimeSim MOSRA User Guide

The *PrimeSim MOSRA User Guide* describes how to set up MOS reliability analysis (MOSRA) and the MOSRA simulation flow.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the PrimeSim MOSRA Release Notes on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the PrimeSim MOSRA tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>

Convention	Description
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN.</code>
 	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Reliability Analysis

This chapter provides an introduction to MOS reliability analysis (MOSRA) and the MOSRA simulation flow.

This chapter covers the following topics:

- [Introduction to the PrimeSim MOSRA Tool](#)
- [PrimeSim MOSRA Simulation Flow](#)
- [MOSRA Supported Models](#)
- [MOSRA Simulation Setup — Examples](#)

Introduction to the PrimeSim MOSRA Tool

The growth in safety-critical applications such as automotive electronics and the continuous scaling down of CMOS technology result in significant increase in IC design reliability challenges. The electrical stress from high electrical field inside transistor, high temperature from device self-heating, ambient temperature, and high frequency switching activities over extended period of time lead to device aging, which is referred as device characteristic degradations. The changes in device characteristics due to electrical stress can impact the circuit performance over a period of time, shorten the circuit lifetime, and introduce potential functional failure in the system. PrimeSim reliability analysis solution provides comprehensive reliability solutions. The PrimeSim MOSRA (MOS Reliability Analysis) tool is one of the products in the PrimeSim reliability family that provide MOS aging modeling and simulation capability from transistor level to full chip. The PrimeSim MOSRA tool, together with PrimeSim EMIR, PrimeSim Custom Fault and PrimeSim AVA simulators, provides a total solution for reliability-aware design and verification.

The PrimeSim MOSRA tool offers accurate and efficient aging simulation capability. With either a built-in or user-defined aging model, the PrimeSim MOSRA tool predicts the important MOSFET reliability aging effects including HCI (hot carrier injection), BTI, (bias temperature instability), and TDDB (time dependent dielectric breakdown) aging effect in both device and circuit levels, enables designers to detect reliability failures early in the design cycle, and significantly reduces the time and cost due to lifetime testing and silicon re-spin.

With tight integration with the PrimeSim Continuumsimulator family, including PrimeSim HSPICE, PrimeSim SPICE, PrimeSim Pro, and PrimeSim XA simulators (“simulator” in short for these products is used in the following contents), the aging analysis can be performed for a full range of designs from the block level to a full chip. In addition, factors such as device aging interaction with process variation and device self-heating effect can be accounted for by various the PrimeSim MOSRA tool commands and options that makes the PrimeSim MOSRA tool a holistic reliability aging analysis solution.

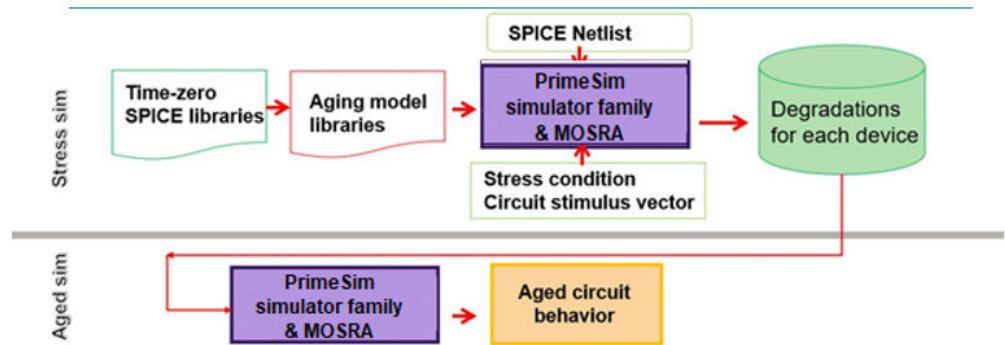
The following sections discuss these topics

PrimeSim MOSRA Simulation Flow

As illustrated in [Figure 1](#), the MOSRA Analysis is a simulation with two phases, the stress simulation phase and the post-stress simulation (that is, the aged simulation) phase. The two phases of the simulation can be setup separately or together:

- During the stress simulation phase, MOSRA computes the electrical stress of the selected MOS transistors in the design, based on circuit behavior and the MOSRA model. The device characteristic change (degradation) is the output of stress simulation.

Figure 1 MOSRA Simulation Flow



- During the aged (or post stress) simulation phase, the circuit figure of merits is simulated based on the degraded MOSFET computed in the stress simulation phase.

MOSRA Supported Models

[Table 1](#) shows the models supported by MOSRA in the PrimeSim HSPICE simulator.

Table 1 Supported Models for MOSRA

Device Model Name	Level
BSIM3	49/53
BSIM4	54
BSIM3SOI	57
RPI Poli-Si TFT	62
HVMOS	66
HiSIM2	68
PSP	69
BSIM4SOI	70
BSIM-CMG	72
HiSIM_HV	73
L-UTSOI	76
BSIM-BULK	77
BSIM-IMG	78
MOSVAR	7
CustomCMI Model	Determined by custom model

Note:

For MOSVAR model, only stress simulation is supported.

For the degradation types supported in each model, refer to [MOSRA API Supported Aging Degradations](#) in Chapter 6.

For more information on the supported model list for MOSRA, see

- *VCS® PrimeSim™ AMS User Guide*
- *PrimeSim™ XA User Guide*

MOSRA Simulation Setup — Examples

The following example demonstrates how to set up a MOSRA simulation using the MOSRA built-in level 3 HCI and BTI model.

```
* MOSRA TEST
vdd 1 0 2
mp1 3 2 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn1 3 2 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
...
c1 2 0 .1p
.model p1 pmos level=54 version=4.5
.model n1 nmos level=54 version=4.5
** MOSRA related setup
.model n1_ra mosra level=3
+BTIFLAG = 1
+BTIAP = 0.01
+BTIM = 3
+HCIFLAG = 1
+HCIAP=1e-9
+HCIM=2
...
.model p1_ra mosra level=3
+BTIFLAG = 1
+BTIAP = 0.01
+BTIM = 3
+HCIFLAG = 1
+HCIAP=1e-9
+HCIM=2
+HCIEA=0.05
...
.appendmodel n1_ra mosra n1 nmos
.appendmodel p1_ra mosra p1 pmos
.mosra reltotaltime=1e8
.ic v(2)=2
.tran .1ps 5ns
.options post
.end
```

The following example demonstrates how to set up a MOSRA simulation using a MOSRA API model.

```
* MOSRA TEST
vdd 1 0 2
mp1 3 2 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn1 3 2 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
...
c1 2 0 .1p
.model p1 pmos level=54 version=4.5
.model n1 nmos level=54 version=4.5
```

```
** MOSRA related setup
.model n1_ra mosra level=101
+ rela=1e-4 relb=2 reln=0.25
.model p1_ra mosra level=101
+ rela=1e-4 relb=2 reln=0.25
.appendmodel n1_ra mosra n1 nmos
.appendmodel p1_ra mosra p1 pmos
.mosra reltotaltime=1e8
.options post mraapi=1
* .option mosrapath to specify the path to the MOSRA API .so library.
* Library path also can be specified by setting environment variable
  hspice_mosra_models
.option mosrapath = "location of libmosra.so"
.ic v(2)=2
.tran .1ps 5ns
.end
```

For more MOSRA examples, see Chapter 5, [Using PrimeSim MOSRA for Aging Analysis](#).

2

PrimeSim MOSRA Reliability Models

The chapter provides information on MOSRA reliability models.

Device aging is a result of continuous degradation of device characteristics, under the applied electrical stress. A MOSRA model is used for translating the amount of electrical stress to the actual device degradation or *age*. Typically, such models are functions of device operating conditions (that is, voltages, currents, and temperature), device geometries, and operation time.

MOSRA supports three types of model. These models are Synopsys built-in MOSRA Level 1 and Level 3 models, and custom reliability-models through the MOSRA API. By choosing the right approach, you can achieve the desired balance between model extraction or implementation effort and design requirements.

The following sections discuss these topics:

- [BTI and HCI Models](#)
- [Custom Reliability Model Using MOSRA API](#)

BTI and HCI Models

Device degradation due to aging can be applied in the following ways:

- The amount of stress is converted to the degradation of key MOSFET compact model parameters, such as threshold voltage (V_{th0}) and mobility (μ_0).

This approach allows for a separation of different effect that contribute to the total device degradation, resulting in better accuracy of current and conductance degradation over a wide range of biases.

The PrimeSim MOSRA built-in Level 1 model uses this approach.

- The stress can be directly converted into device characteristic degradation, for example, MOSFET channel current change.

This approach has the advantage of simplicity, especially because the aging model can be generated directly from the hardware data.

The PrimeSim MOSRA built-in Level 3 model uses this approach.

This second approach has the advantage of simplicity, especially because the aging model can be generated directly from the hardware data. On the other hand, the first approach allows for a separation of different effects that contribute to the total device degradation, resulting in better accuracy of current and conductance degradation over a wide range of biases. MOSRA built-in Level 1 and Level 3 models use the first and approaches, respectively.

Level 1 BTI and HCI Model

The MOSRA Level 1 BTI and HCI model represents the aging effect through compact model parameters. The model capability is best utilized when sweep measurement of an entire family of I-V curves of an aged MOS transistor is available for parameter extraction.

The mathematical formulation of this model and its detailed model parameter list are accessible through a non-disclosure agreement (NDA). For more information, contact the Synopsys technical support team.

Level 3 BTI and HCI Model

The MOSRA Level 3 BTI and HCI model is a concise and efficient model proven by many designs. It can adequately fit to the device's electrical parameter degradation (for example, I_{dlin} and V_{th} of an aged MOS transistor) under various bias conditions.

BTI Model

Use the model parameter $BTIFLAG=0|1$ to disable/enable the BTI model (default = 1).

V_{th} at $EOL = V_{th}(TO) + V_{th_{shift}}(BTI)$ where $V_{th}(TO) = type^* V_{on}$. Here V_{on} is the device real threshold voltage obtained from the compact model.

BTI degradation equation (unit of mV):

$$V_{th_{shift}} = A_p \cdot e^{G \times type \times V_{gs}} \cdot e^{(-Ea)/(kT)} \cdot t_{stress}^n \cdot e^{-D \times abs(V_{ds})} \cdot e^{mL}$$

Where:

- T is the channel temperature in Kelvin
- k is the Boltzmann's constant (8.617×10^{-5} eV/K)
- L is the polysilicon length of the device in μm

- V_{ds} is the drain to source voltage in Volts
- V_{gs} is the gate to source voltage in Volts
- t is the total stress time in seconds
- $type$ is MOSFET device type

BTI Partial recovery effect model:

$$V_{th_{shiftAC}} = V_{th_{shift}} \cdot \exp(-BTIR \times g)$$

Where g is the duty cycle defined as:

$$g = 1 - \frac{\int_0^t (V_{gs} \cdot type) dt}{t \cdot V_{GS_{MAX}}}$$

Synopsys LEVEL 3 BTI Model Parameters

Table 2 V_{th} Degradation BTI Parameters

Name	Default	Description	Notes
BTIFLAG	1	BTI model selector	Disables/enables the BTI model
BTIAP	0.0	Pre-factor	
BTIM	3.0	L dependency	
BTIEA	0.143	Temperature dependency	
BTID	0.0	V_{ds} voltage dependency	
BTIG	5.0	V_{gs} voltage dependency	
BTIN	0.175	Time dependency	
BTIR	0.5	Recovery parameter	
VGSMAX	1.5	VGSMAX	See item 1 in the Note following the table
BTILFLAG	0	L selection flag	See item 2 in the Note following the table

Note:

1. VGS MAX should be positive. If VGS MAX ≤ 0, set g=0; For VGS MAX usage: if (VGS MAX is specified), then VGS MAX = VGS MAX from the model card; otherwise, VGS MAX = max(Vgs*type).
2. BTI equation uses the BTILFLAG model parameter in Leff/Ldrawn selection. BTILFLAG = 0: default, uses Leff. BTILFLAG = 1: uses Ldrawn.

HCI Model

Use the model parameter HCIFLAG=0|1 to disable/enable HCI model (default = 1).

The IDS at EOL = IDS (T0) * (1 -Ids_{shift}%)

HCI Degradation Equation:

$$I_{ds_{shift}}(\%) = A_p \cdot e^{G_{type} \times V_{gs}} \cdot e^{\frac{-E_a}{kT}} \cdot t_{stress}^n \cdot e^{D \times abs(V_{ds})} \cdot e^{-mL} Switch$$

Where:

- *Switch* = if (Vgs < Vgsmin then 0 else 1)
- *T* = channel temperature in Kelvin
- *k* = Boltzmann's constant (8.617 × 10⁻⁵ eV/K)
- *L* = polysilicon length of the device in μm
- *Vds* = Drain to source voltage in Volts
- *Vgs* = Gate to source voltage in Volts
- *t* = total stress time in seconds
- *type* is MOSFET device type

Synopsys LEVEL 3 HCI Model Parameters

Table 3 *Ids Degradation HCI Parameters*

Name	Default	Description	Notes
HCIFLAG	1	HCI model selector	Disables/enables the HCI model
HCIAP	0.0	Pre-factor	
HCIM	2.0	L dependency	
HCIEA	0.05	Temperature coefficient	

Table 3 *Ids Degradation HCI Parameters (Continued)*

Name	Default	Description	Notes
HCID	9.0	Vds voltage dependency	
HCIG	0.0	Vgs voltage dependency	
HCIN	0.5	Time dependency	
VGSMIN	0.2	VGSMIN	See item 1 in the Note following the table
HCILFLAG	0	Flag of L selection	See item 2 in the Note following the table

Note:

1. For VGSMIN usage: if VGSMIN is specified in the model card, use the given value; else, use V_t (the threshold voltage).
2. HCI equation uses the HCILFLAG model parameter in I_{eff}/I_{drawn} selection. HCILFLAG = 0: default, uses I_{eff} . HCILFLAG = 1: uses I_{drawn} .

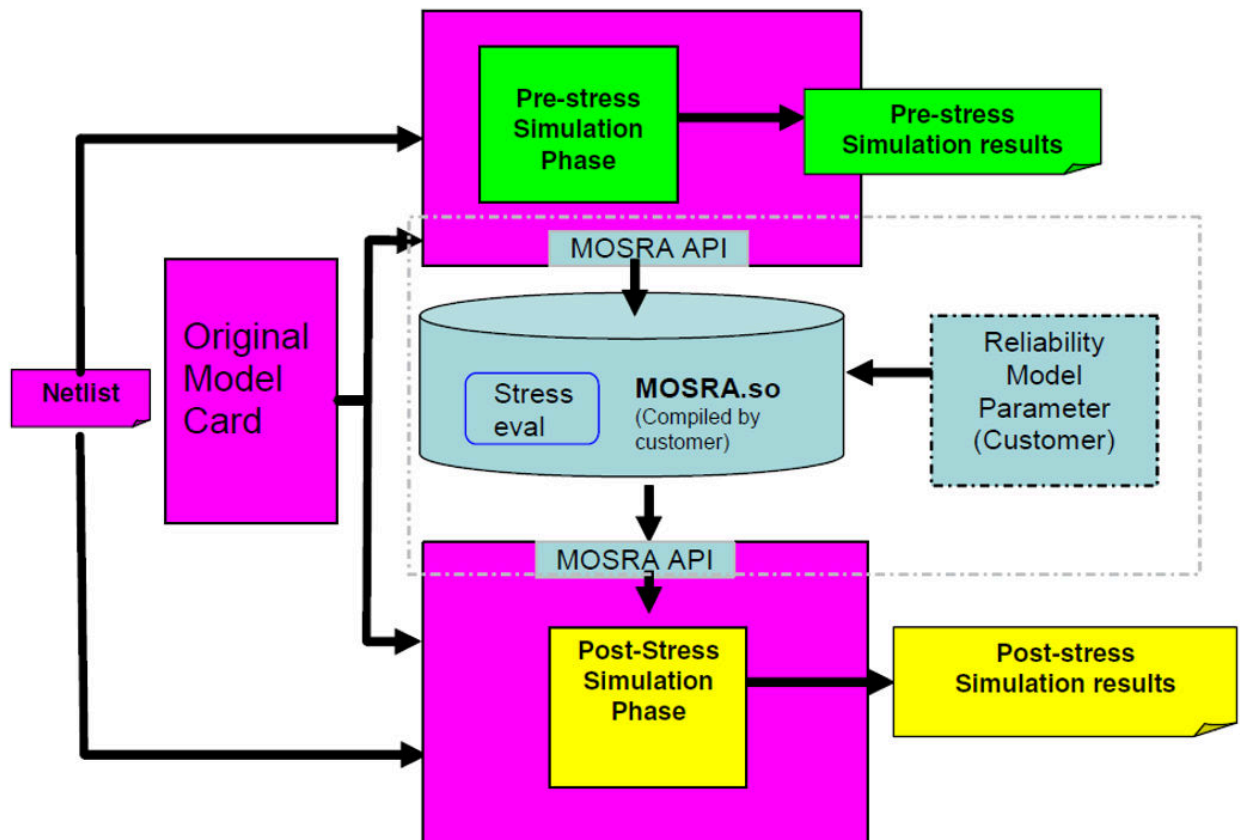
Clamping: If HCI degradation > 90%, the value is clamped to 90%.

Custom Reliability Model Using MOSRA API

For users who develop their own reliability aging models, MOSRA API is provided to deploy custom aging models, and continue to use the same MOSRA simulation engine as the built-in BTI and HCI models.

The MOSRA API is C-language modeling API; you can implement your own model equations through the API, and compile it into a Dynamically Linked Library (DLL). It provides the flexible platform for formulating MOSFET aging equations and stress computation algorithms. It allows you to develop and implement full custom MOSFET aging models as well as custom stress integration/prediction algorithms. MOSRA API model and MOSRA built-in model share the same simulation engine and flow as shown in [Figure 2](#).

Figure 2 MOSRA Flow with API Access



For more detailed information on MOSRA API, see [MOSRA API for Custom Aging Model Development](#).

3

PrimeSim MOSRA Simulation Commands

This chapter provides a listing of the PrimeSim MOSRA netlist commands.

This chapter provides the list of PrimeSim MOSRA netlist commands and detailed descriptions of the individual commands.

- [.MOSRA](#)
- [.MOSRAPRINT](#)
- [.MOSRA_SUBCKT_PIN_VOLT](#)
- [.MODEL](#)
- [.APPENDMODEL](#)

.MOSRA

Invokes HCI and/or BTI reliability analysis.

Syntax

```
.MOSRA RelTotalTime=time_value [RelStartTime=time_value]
+ [DEC=value] [LIN=value] [LOG=value]
+ [RelTimePoints=(time_val_1 [time_val_2] ... [time_val_n])]
+ [RelStep=time_value] [RelMode= 0|1|2] [SimMode= 0|1|2|3|4]
+ [AgingStart=time_value] [AgingStop=time_value]
+ [AgingPeriod=time_value] [AgingWidth=time_value]
+ [AgingInst="inst_name"]
+ [AgingInstExcept="inst_name"]
+ [Integmod= 0|1|2] [Xpolatemod= 0|1|2]
+ [Tsample1=value] [Tsample2=value]
+ [Agethreshold=value]
+ [MosraLife=degradation_type_keyword] [DegF=value]
+ [DegFM="ModelName1=value1 [ModelName2=value2 ...]" ]
+ [DegFN=value] [DegFP=value]
+ [Frequency=value]
+ [hci= 0|1] [bti= 0|1] [tddb= 0|1]
+ [circuit_report= 0|1] [trelax=value] [area_scaling=value]
+ [earlyExitEnabled =0|1]
```

```
+ [postAgeSim = 0|1|2]
+ [RelTimeRatio="ratio_information_string"]
```

Arguments

Argument	Description
RelTotalTime	Final reliability test time to use in post-stress simulation phase. Required argument where <i>time_value</i> can be in units of: • sec (default with no unit entry required) • min • hr • day • yr
RelStartTime	Time point of the first post-stress simulation. Default is 0.
DEC	Specifies the number of stress time points simulated per decade.
LIN	Linear stress time points from RelStartTime to RelTotalTime.
LOG	Specifies the number of stress time points simulated in log scale. The stress time points are calculated using the following equation: $\exp \left(index \times \frac{\ln (1.0 + RelTotalTime)}{LOG} \right) - 1.0$ Where index=1,2,...,LOG
RelTimePoints	• Specify user-defined time points for post-stress simulation. The required arguments (<i>time_val_1</i> [<i>time_val_2</i>] ... [<i>time_val_n</i>]) should follow these rules: • Can be in units of: sec (default, with no unit entry required), or min, or hr, or day, or yr. • The first time point value should be greater than RelStartTime, if RelStartTime is specified; the last time point value should be less than and equal to RelTotalTime.
RelStep	Post-stress simulation phase on time= RelStep, 2* RelStep, 3* RelStep, ... until it achieves the RelTotalTime; the default is equal to RelTotalTime. Value is ignored if DEC or LIN or LOG, or RelTimePoints is set.

Argument	Description
RelMode	Synopsys simulator built-in level 1 model selects whether a simulation accounts for both HCI and BTI effects or either one of them. If the RelMode in the .MOSRA command is defined as 1 or 2, it takes higher priority and applies to all MOSRA models. If RelMode in the .MOSRA command is not set or set to 0, then the RelMode inside individual MOSRA models take precedence for that MOSRA model only; the rest of the MOSRA models take the RelMode value from the .MOSRA command. If any other value is set, except 0, 1, or 2, a warning is issued, and RelMode is automatically set to the default value 0. • 0: both HCI and BTI, Default • 1: HCI only • 2: BTI only
SimMode	• 0: Select pre-stress simulation only • 1: Select post-stress simulation only • 2: Select both pre- and post-stress simulation, Default • 3: Select gradual aging mode 1, continual stress integration through .ALTERs • 4: Select gradual aging mode 2, custom aging accumulation When SimMode=1 • Simulator reads in the *.radeg0 file and uses it to update the device model for reliability analysis; new transient output file is generated as the *.tr@ra=xxx. • The netlist stimuli could be different from the SimMode=0 netlist that generated the *.radeg file. When SimMode=3 • If you do not specify .option radegfile in the top-level netlist, the simulation starts from a fresh device. • If you specify .option radegfile in the top-level netlist, simulator reads in the last suite degradation from the radeg file, and continues the degradation integration/extrapolation from the corresponding circuit time in the radeg file. • In consecutive alters, simulator reads in the radeg generated from the previous .ALTER run.

Argument	Description
	When <code>SimMode=4</code>: • If you do not specify <code>.option radegfile</code>, it is equivalent to <code>SimMode=0</code> and the simulation starts afresh. • If you specify <code>.option radegfile</code>, simulator reads back in the last suite degradation in the <code>.radeg</code> file and uses it to update the device model for reliability analysis. Aging computation and integration for the current simulation starts from an initial degradation value being 0, and the aging extrapolation and output is based on the operation time interval specified (<code>reltotaltime</code>) in the current netlist. The operation time in the <code>.radeg</code> output file is the time interval of current simulation as well. Note: For the PrimeSim HSPICE simulator you can use the command-line option <code>-mrasm</code> to overwrite the value of <code>SimMode</code> in a <code>.MOSRA</code> command card. <code>Hspice input.sp -mrasm [0 1 2 3 4]</code>
<code>AgingStart</code>	Optionally defines time when MOSRA starts stress effect calculation during transient simulation. Default is 0.0. See Example 1 , Example 2 , Example 3 .
<code>AgingStop</code>	Optionally defines time when MOSRA stops stress effect calculation during transient simulation. Default is <code>tstop</code> in <code>.TRAN</code> command. See Example 1 , Example 2 , Example 3 .
<code>AgingPeriod</code>	Stress period. Scales the total degradation over time. See Example 1 , Example 2 , Example 3 .
<code>AgingWidth</code>	The <code>AgingWidth</code> (circuit time “on”) argument works with the <code>AgingPeriod</code> argument. For example: if you specify <code>AgingPeriod=1.0s</code> and <code>AgingWidth=0.5s</code> , then the circuit is turned on for 0.5s, and turned off for 0.5s. (The period is 1.0s.). See Example 1 , Example 2 , Example 3 .
<code>AgingInst</code>	Selects MOSFET devices, which MOSRA applies aging analysis. The default is all MOSFET devices with reliability model appended. The name must be surrounded by quotation marks. Multiple names in multiple lines are allowed and wildcards are supported. Example 4 .
<code>AgingInstExcept</code>	Use this argument to deselect instances for MOSRA analysis. Multiple names in multiple lines are allowed and wildcards are supported. See Example 5 .
<code>Integmod</code>	Flag to select the integration method and function. • 0 (default): User-defined integration function in MOSRA API • 1: Linearized integration method (support non-constant n coefficient) • 2: Derivation-based integration method

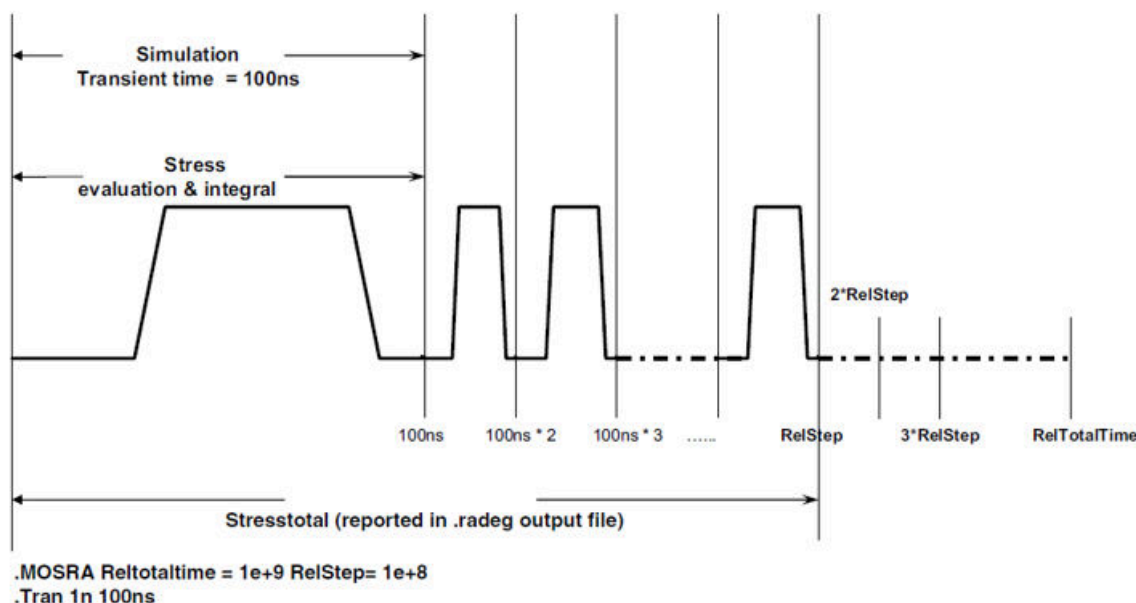
Argument	Description
Xpolatemod	Flag to select the extrapolation method and function. • 0 (default): User-defined extrapolation function in MOSRA API • 1: Linearized extrapolation method (support non-constant n coefficient) • 2: Two-point fitting-based extraction and extrapolation method
Tsample1	First simulation time point of stress_total sampling for Xpolatemod=2.
Tsample2	Second simulation time point of stress_total sampling for Xpolatemod=2.
Agethreshold	Only when the degradation value $\geq$ Agethreshold, the MOSFET information is printed in the MOSRA output file *.radeg or *.cvs file. Default is -1e+100. See Example 4 .
MosraLife	Argument to compute device lifetime for the degradation type specified. This argument has the same function as .OPTION MOSRALIFE. If .OPTION MOSRALIFE is specified, it takes precedence over .MOSRA MOSRALIFE.
DegF	Sets the MOSFET's failure criteria for lifetime computation. This argument has the same function as .option degf. If .option degf is specified, it takes precedence over .MOSRA DegF.
DegFM	Sets the model-based failure criteria for lifetime computation. This argument has the same function as .OPTION DEGFM. If the criteria for the same model is also specified in .OPTION DEGFM, it takes precedence over the value set in .MOSRA DEGFM. See more details in .OPTION DEGFM.
DegFN	Sets the NMOS's failure criteria for lifetime computation. This argument has the same function as .option degfn. If .option degfn is specified, it takes precedence over .mosra degfn.
DegFP	Sets the PMOS's failure criteria for lifetime computation. This argument has the same function as .option degfp. If .option degfp is specified, it takes precedence over .mosra degfp.
Frequency	User-specified frequency of the signal for BTI frequency-dependent recovery effect calculus. If not specified, the value is automatically calculated. This command is for built-in level 1 model only.
hci	Control flag to invoke HCI model in API. • 0 (Default): HCI off • 1: on
bti	Control flag to invoke BTI model in API. • 0 (Default): BTI off • 1: on

Argument	Description
<code>tddb</code>	Control flag to invoke TDDb model in API. • 0 (Default): TDDb off • 1: on
<code>circuit_report</code>	Control flag to indicate if there is user-defined aging report to be generated in the MOSRA API model. • 0 (Default): no report • 1: yes
<code>trelax</code>	User-defined relaxation time for models in API. Default is 0.0.
<code>area_scaling</code>	User-defined area scaling coefficient for models in API. Default is 1.0.
<code>earlyExitEnabled = 0 1</code>	Flag used in MOSRA API to determine whether the gradual aging flow based on <code>simmode=2</code> with multiple <code>relSteps</code> requires early-exit. • 0: Custom aging model (MOSRA API) detects early-exit, default value • 1: Custom aging model (MOSRA API) detects early-exit Note: MOSRA simulation jumps to the last <code>relStep</code> when <code>POST_exitGradualAging(value = 1)</code> flag is received from the MOSRA API model. Valid only for <code>multiple-RelStep SimMode=2</code> flow. For other <code>SimMode</code> flows, the early-exit flag is ignored.
<code>postAgeSim = 0 1 2</code>	Flag to determine whether post-stress simulation is skipped In <code>SimMode=2</code> with multiple <code>relSteps</code> (.MOSRA DEC/LIN/LOG/RelStep/RelTimePoints) flow: • 0: skips the last post-stress simulation • 1: (default) runs all post-stress simulations • 2: not supported In <code>SimMode=3</code> with <code>.alters</code> flow. • 0: only runs stress simulation in each alter except the last one. The last alter runs both stress and post-stress simulations. Default • 1: run both stress and post-stress simulations in each alter • 2: only runs stress simulation in each alter

Argument	Description
RelTimeRatio	Specify a ratio for certain blocks/instances to apply a stress time which is different with RelTotalTime. - The input string is a command or a file name - If the input is a command, it follows: <pre>.MOSRA RelTimeRatio = "blk/inst_name1:ratio1 blk/inst_name2:ratio2 ..."</pre> - blk/inst_name represents block/instance name, wildcard is supported - Ratio is a number, represents the stress time ratio for given block/instance, value should be > 0 - Name and ratio are separated by ":" in each record - Multiple commands are supported, separated by blank or tab - If the input is a file name, simulator opens this file and read in stress time ratio command. File format is as follows: - blk/inst_name1:ratio1 blk/inst_name2:ratio2 - blk/inst_nameN:ratioN - Multiple commands defined in a single line is supported, separated by blank or tab - One command defined in multiple lines is not permitted. For details, see Example 6

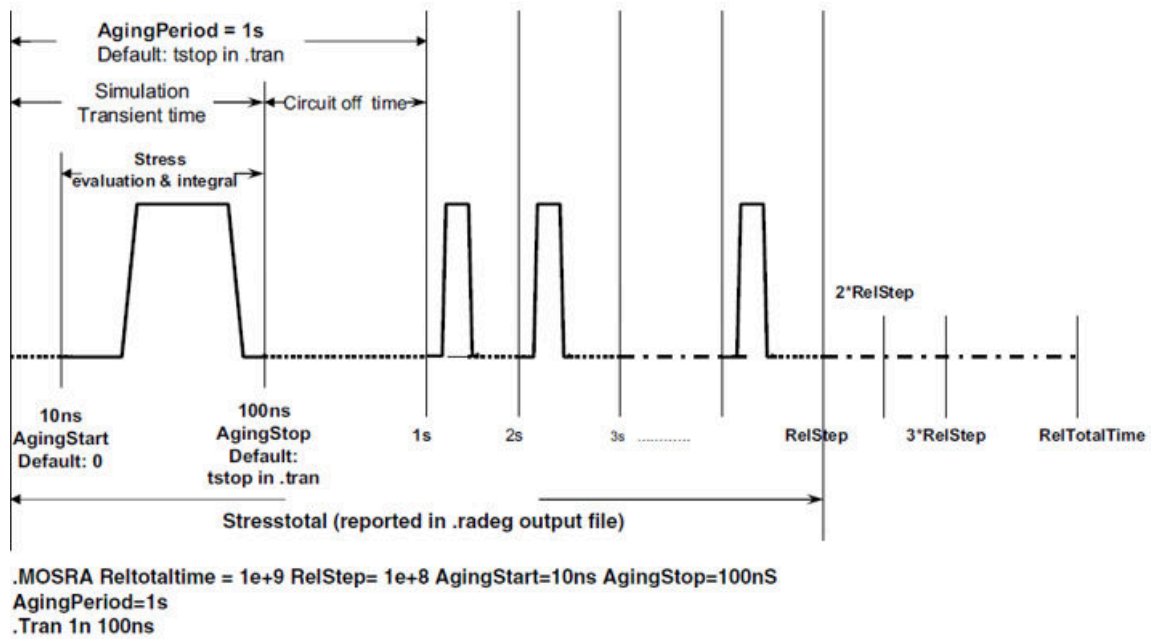
Example 1

Command/syntax for Reliability Simulation (1/3)



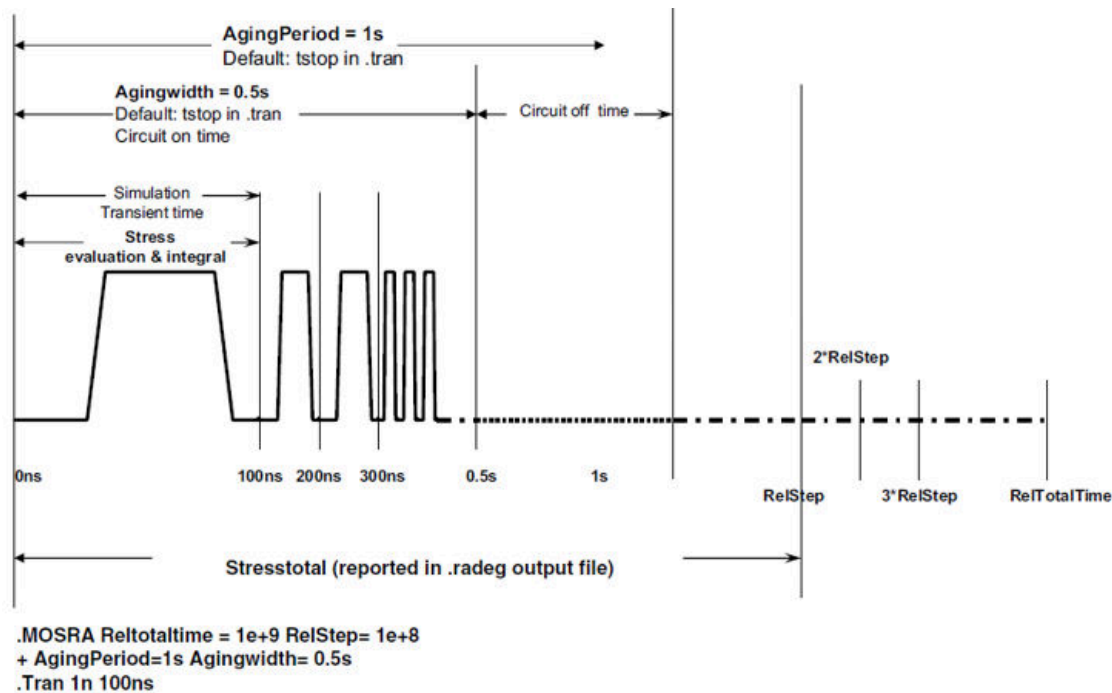
Example 2

Command/syntax for Reliability Simulation (2/3)



Example 3

Command/syntax for Reliability Simulation (3/3)



This example does simmode=2 analysis for total age time of 6.3×10^8 seconds with relstep of 6.3×10^7 seconds. The MOSRA analysis is applied to MOSFETs with instance name starting from x1. The MOSRA stress calculation starts from 5ns and stops at 100ns.

```
.mosra reltotaltime=6.3e+8 relstep=6.3e+7
+ agingstart=5n agingstop=100n
+ aginginst="x1.*"
```

Example 4

simmode=2 Analysis

This example does simmode=2 analysis for total age time of 6.3×10^8 seconds. Devices with degradation value ≥ 0.1 is printed in *.radeg file and *.radeg.csv file.

```
.options post
.mosra reltotaltime=6.3e+8 agethreshold = 0.1
.option radegoutput=csv
```

Example 5

AgingInst and AgingInstExcept Usage Examples

```
mp1 3 2 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn1 3 2 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp2 4 3 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn2 4 3 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp3 2 4 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
```

```
mn3 2 4 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
* mosra command
.mosra reltotaltime=1e+4
+ aginginst="mn1 mn2
+ mn3"
```

or

```
* mosra command
.mosra reltotaltime=1e+4
+ aginginstexcept="mp1
+ mp2
+ mp3"
```

Example 6

RelTimeRatio Usage Examples

```
.MOSRA Reltotaltime=5yr RelTimeRatio="mn*:0.5 mp*:0.8 X1.*:0.6"
```

Stress time is $5 * 0.5 = 2.5\text{yr}$ for devices with names starting with mn; stress time is $5 * 0.8 = 4\text{yr}$ for devices whose name start with mp; stress time is $5 * 0.6 = 3\text{yr}$ for devices in block X1.

or

```
.MOSRA Reltotaltime=5yr RelTimeRatio="./stresstime_file"
In stresstime_file:
mn*:0.5 mp*:0.8
X1.*:0.6
```

These two examples are equivalent on RelTimeRatio usage.

.MOSRAPRINT

Provides .PRINT/.PROBE capability for device electrical degradations.

Syntax

```
.MOSRAPRINT output_name output_type(element_name, vds=exp1, vgs=exp2,
vbs=exp3)
```

Arguments

Argument	Description
output_name	User-defined output variable; this <code>output_name@element_name</code> is used as the as output variable name in the output file.
output_type	One of the following output variable types: vth, gm, gds, ids, dids or dvth

Argument	Description
<i>element_name</i>	The element name that the .MOSRAPRINT command applies.

Description

The .MOSRAPRINT command supports the following models: B3SOI, B4SOI, PSP, BSIM3, BSIM4, TFT, HVMOS, HiSIM-HV, UTSOI, BSIM-CMG, BSIM-BULK, BSIM-IMG, and Custom CMI MOSFETS.

This command provides access to device degradation information. The *vds*, *vgs* and *vbs* are user-specified bias conditions used to characterize the device electrical property as specified by the output type. There is no order requirement for *vds*, *vgs*, and *vbs*. Wildcards '?' and '*' are supported in *element_name*. The output variable *dids* reports the percent change of *ids* between post-stress simulation and fresh-device. *dvth* reports the change of *vth* between post-stress simulation and fresh-device. The output file format is the same as the measurement file format with file extension *.ra. You can use .OPTION MEASFORM with this command to produce *.csv files suitable for Microsoft Excel output.

.MOSRAPRINT does approximate calculation to get the *ids*.

Examples

The following syntax prints the *ids* value of the MOSFET *m1*, when *vds* = 5*vgs*=5, *vbs*=0, at each reltime point.

```
.MOSRA reltotaltime=5e+7 relstep=1e+7
.MOSRAPRINT idsm1 ids(m1, vds=5, vgs=5, vbs=0)
```

.MOSRA_SUBCKT_PIN_VOLT

When a MOSFET is wrapped by a subcircuit-based macro model, this command specifies the subcircuit terminal voltages used by MOSRA model evaluation.

Syntax

```
.MOSRA_SUBCKT_PIN_VOLT subckt_name1, subckt_name2,...
```

Description

Use this command to specify subcircuit-based macro terminal voltages the simulator uses for MOSRA model evaluation.

subckt_name1, ...: The subcircuit names whose terminal voltages are used for MOSRA model evaluation.

Note:

There is a limitation to this capability. The subcircuit-based macro model can contain only one MOSFET, and the number and definition of subcircuit terminals must be consistent with MOSFET terminal number and definition.

Examples

In this example, the simulator uses subcircuit sub1's terminal voltages $v(d)/v(g)/v(s)/v(b)$, instead of the MOSFET M1's terminal voltages, $v(d1)/v(g1)/v(s1)/v(b1)$ for MOSRA model evaluation.

```
.subckt sub1 d g s b ...
M1 d1 g1 s1 b ...
Rd d d1 1k
Rs s s1 1k
Rg g g1 1k
.model ...
.ends
.mosra_subckt_pin_volt sub1
...
.end
```

.MODEL

```
.model mname mosra
+ level|mosralevel=value
+[relmodelparam]
```

Argument	Description
mname	User-defined MOSFET reliability model name, supports binning model definition.
mosra	MOSRA model primitive.keyword.
level (alias: mosralevel)	To use the Synopsys MOSRA model, set <i>LEVEL=1</i> or <i>LEVEL=3</i> . For compatibility in the in the .MODEL statement, 'LEVEL=' can be replaced with 'MOSRALEVEL='. Simulator considers them equivalent. Example: The following two lines are interpreted the same by simulator: .MODEL my_mod MOSRA LEVEL=1 .MODEL my_mod MOSRA MOSRALEVEL=1 For custom MOSRA models, use the level implemented in the MOSRA API model.
relmodelparam	Reliability model parameters.

.APPENDMODEL

Appends the .MOSRA (model reliability) parameters to a *fresh* model card.

Syntax

```
.APPENDMODEL SrcModel ModelKeyword1 DestModel ModelKeyword2
```

Arguments

Argument	Description
<i>SrcModel</i>	Source model name, for example, the name of the MOSRA model.
<i>ModelKeyword1</i>	Model type for <i>SrcModel</i> . For example, the keyword <code>mosra</code> .
<i>DestModel</i>	Destination model name, for example: - The original model in the model library. - The model name with full hierarchy of subcircuit definition in the model library: <code><sub_def1>::<sub_def2>::...::<model_name></code>
<i>ModelKeyword2</i>	Model type for <i>DestModel</i> . For example, <code>nmos</code> .

Description

Appends the parameter from the source model card (*SrcModel*) to the destination model card (*DestModel*). All arguments are required. Wildcards and binning model are supported for the .APPENDMODEL command. In addition, the .OPTION APPENDALL enables the top hierarchical level to use the .APPENDMODEL command even if the MOSFET model is embedded in a subcircuit. The .OPTION APPENDALL command also enables the top hierarchical level to use the .APPENDMODEL command to append different MOSRA models to different subcircuit macro models.

Examples

Example 7 Model Card hci_1 to the b3_nch BSIM3

This examples appends the content of the model card hci_1 to the b3_nch BSIM3 model card.

```
.appendmodel hci_1 mosra b3_nch nmos
```

Example 8 Model Card hci_1 to the mosvar_cap MOS Varactor

Appends the content of the model card hci_1 to the mosvar_cap MOS Varactor model card. (Currently for capacitor devices, only MOS Varactor model is supported.)

```
.appendmodel hci_1 mosra mosvar_cap c
```

Example 9 *Model p1_ra to the pmos Models.*

Model p1_ra is appended to all of the pmos models. Quotation marks are required if the model name is defined only by a wildcard.

```
.appendmodel p1_ra mosra "*" pmos
```

Example 10 *Model p1_ra to all of the pmos Models*

The model p1_ra is appended to all of the pmos models that are named pch* (pch1, pch2, pch_tt, and so on).

```
.appendmodel p1_ra mosra pch* pmos
```

Example 11 *MOSFET Binning Model Usage*

```
* MOSFET binning model
.model nch.0 nmos level=54
+ lmin=lmin0 lmax=lmax0
+ wmin=wlim0 wmax=wlim1
+ ...
.model nch.1 nmos level=54
+ lmin=lmin0 lmax=lmax0
+ wmin=wlim1 wmax=wlim2
+ ...
.model nch.2 pmos level=54
+ ...
...

* MOSRA model
.model nch_ra mosra level=101
+ parameter1 = value1 ...

* APPENDMODEL command
.appendmodel nch_ra mosra nch nmos
```

Note:

- You can only specify the master name (without .#) of binning models for *DestModel* in the .APPENDMODEL command.
- The MOSRA model is appended to each MOSFET bin model, that is, nch_ra appended to nch.1, nch.2,...

Example 12 *MOSFET and MOSRA Binning Model Usage*

```
* MOSFET binning model
.model nch.0 nmos level=54
+ lmin=lmin0 lmax=lmax0
+ wmin=wlim0 wmax=wlim1
+ ...
```

Chapter 3: PrimeSim MOSRA Simulation Commands

.APPENDMODEL

```
.model nch.1 nmos level=54
+ lmin=lmin0 lmax=lmax0
+ wmin=wlim1 wmax=wlim2
+ ...
.model nch.2 pmos level=54
+ ...
...

* MOSRA binning model - no geometry information
.model nch_ra.0 mosra level=101
+ parameter1 = value1 ...
.model nch_ra.1 mosra level=101
+ parameter1 = value1 ...
.model nch_ra.2 mosra level=101
+ parameter1 = value1 ...
...

* APPENDMODEL command
.appendmodel nch_ra mosra nch nmos
```

Note:

- The # of MOSFET bin and the # of MOSRA model bins must be the same.
- You can only specify master name (without .#) of binning models for *srcModel* and *DestModel* in the .APPENDMODEL command.
- The bin selection is conducted only for the MOSFET model; no bin selection on the MOSRA model.
- The MOSRA bin model is 1-to-1 appended to the MOSFET bin model, that is, nch_ra.1 is appended to nch.1, nch_ra.2 is appended to nch.2... .

Example 13 Append Different MOSRA Models to Subcircuit Macro Models

Model nch_ra_1 is appended to nmos model card nch defined in subcircuit mosra_test_1, model nch_ra_2 is appended to nmos model card nch defined in subcircuit mosra_test_2.

```
.option appendall
.appendmodel nch_ra_1 mosra mosra_test_1::nch nmos
.appendmodel nch_ra_2 mosra mosra_test_2::nch nmos
.model nch_ra_1 mosra ...
.model nch_ra_2 mosra ...
.SUBCKT mosra_test_1 1 2 3 4
M1 1 2 3 4 nch L=PL W=PW
.model nch nmos level=...
.ENDS mosra_test_1
.SUBCKT mosra_test_2 1 2 3 4
M1 1 2 3 4 nch L=PL W=PW
.model nch nmos level=...
.ENDS mosra_test_2
```

Note:

- This usage is limited to use the `.APPENDMODEL` commands in top hierarchical level and the `.OPTION APPENDALL` command must be set.
- If a subcircuit macro model card is specified in the two `.APPENDMODEL` commands of top hierarchical level, the command with full hierarchical subcircuit macro model name takes a higher precedence.

4

MOSRA Control Options Reference

Presents reference information for each of the PrimeSim MOSRA control options.

This chapter provides the list of PrimeSim MOSRA control options and detailed descriptions of the individual control options.

- [.OPTION APPENDALL](#)
- [.OPTION DEGF](#)
- [.OPTION DEGFN](#)
- [.OPTION DEGFP](#)
- [.OPTION DEGFM](#)
- [.OPTION MRA_POST_COMPUTE](#)
- [.OPTION MOSRALIFE](#)
- [.OPTION MOSRAPATH](#)
- [.OPTION MOSRASORT](#)
- [.OPTION MOSRASORTMODE](#)
- [.OPTION MRAAPI](#)
- [.OPTION MRADTEMPBA](#)
- [.OPTION MRAEXT](#)
- [.OPTION MRAMCFEIL](#)
- [.OPTION MRAPAGED](#)
- [.OPTION MRA_CONTROL](#)
- [.OPTION MRASHETEMP](#)
- [.OPTION MRA00PATH, MRA01PATH, MRA02PATH, MRA03PATH](#)

- [.OPTION RADEGFILE](#)
- [.OPTION RADEGOUTPUT](#)

.OPTION APPENDALL

Allows the [.APPENDMODEL](#) command to append the MOSRA model to the MOSFET model card, if this MOSFET model definition is in a subcircuit.

Syntax

```
.OPTION APPENDALL
```

Description

When the MOSFET model is defined in the current hierarchical level, the [.APPENDMODEL](#) command can append the MOSRA model parameters to the MOSFET model card.

If the MOSFET model is defined in a subcircuit, with `.OPTION APPENDALL`, the [.APPENDMODEL](#) command appends the MOSRA model parameters to this MOSFET model card. Otherwise, a warning message similar to the following is issued:

```
Cannot find model NFET at the same level of .appendmodel command. Verify  
that it is defined or used
```

If the [.APPENDMODEL](#) commands are provided in both the current hierarchical level and in a subcircuit, the [.APPENDMODEL](#) in the subcircuit takes a higher precedence.

Examples

Example 14 Append the n_ra MOSRA to nch MOSFET Model

In this example, the n_ra MOSRA model parameters are appended to the nch MOSFET model card.

```
.option appendall  
.appendmodel n_ra mosra nch nmos  
  
.model n_ra mosra ...  
  
.SUBCKT mosra_test 1 2 3 4  
M1 1 2 3 4 nch L=PL W=PW  
.model nch nmos level= ...  
.ENDS mosra_test
```

Example 15 Append the n_ra_sub MOSRA to nch MOSFET Model

In this example, the n_ra_sub MOSRA model parameters are appended to the nch MOSFET model card.

```
.option appendall
.appendmodel n_ra mosra nch nmos

.model n_ra mosra ...
.model n_ra_sub mosra ...

.SUBCKT mosra_test 1 2 3 4
M1 1 2 3 4 nch L=PL W=PW
.model nch nmos level= ...
.appendmodel n_ra_sub mosra nch nmos
.ENDS mosra_test
```

.OPTION DEGF

Sets the device's failure criteria for lifetime computation when using the MOSRA API if no values are set for [.OPTION DEGFn](#) or [.OPTION DEGFP](#).

Syntax

```
.OPTION DEGF=val
```

Description

This option is used in conjunction with [.OPTION MOSRALIFE](#). For NMOS, `DEGFn` is used. If `DEGFn` is not defined, `DEGF` is used instead.

For PMOS, `DEGFP` is used. If `DEGFP` is not defined, `DEGF` is used instead. This option sets the device's degradation value at lifetime. The options apply to all MOSFETs. The lifetime values are printed in the RADEG file.

.OPTION DEGFn

Sets the NMOS's failure criteria for lifetime computation when using the MOSRA API.

Syntax

```
.option DEGFn=val
```

Description

This option is used in conjunction with [.OPTION MOSRALIFE](#). This option sets the NMOS's degradation value at lifetime. If this option is not specified or the keyword can not be identified by the `MRAlifetimeDeg` function, simulator substitutes [.OPTION DEGF](#) for lifetime computation. The options apply to all NMOS MOSFETs. The lifetime values are printed in the RADEG file.

.OPTION DEGFP

Sets the PMOS's failure criteria for lifetime computation when using the MOSRA API.

Syntax

```
.option DEGFP= val
```

Description

This option is used in conjunction with [.OPTION MOSRALIFE](#). This option sets the PMOS's degradation value at lifetime. If this option is not specified or the keyword cannot be identified by the MRAlifetimeDeg function, the simulator substitutes [.OPTION DEGF](#) for lifetime computation. The options apply to all PMOS MOSFETs. The lifetime values are printed in the RADEG file.

.OPTION DEGFM

Sets the model-based failure criteria for lifetime computation.

Syntax

```
.OPTION DEGFM= "ModelName1=value1 [ModelName2=value2 ...]"
```

Description

This option is used in conjunction with [.OPTION MOSRALIFE](#). It allows you to set the model-based degradation value at lifetime.

It has high priority when used together with the `degfn/degfp` and `degf` options. The simulator attempts to find the criteria from the `degfm` option first. If the `degfm` option is not set or model criteria is not set in the `degfm` option, the simulator uses the value of the `degfn/degfp` option or the `degf` option (if `degfn/degfp` is not set either).

The criteria for the arguments are:

- Input must be surrounded by quotation marks
- `ModelName#` represents the device compact model name. Wildcard is supported
- Macro model name is not supported
- Model name and value are separated by “=” in each record
- Multiple records are supported in one option, separated by blank or tab
- Multiple `degfm` options in one netlist are supported

.OPTION MRA_POST_COMPUTE

Enables access to MOSRA API function `MRApostCompute`.

Syntax

```
.OPTION MRA_POST_COMPUTE = [0|1]
```

Default If option is not specified in the netlist: 0; if option name is specified without a corresponding value: 1

Description

Use this option to access to MOSRA API function `MRApostCompute`.

The amount of Ids degradation is MOSFET operation region dependent. When `.OPTION MRA_POST_COMPUTE=1`,

MOSRA API provides a function **`MRApostCompute`** that allows you to scale Ids degradation with respect to MOSFET bias condition. This function takes effect during postgate simulation.

.OPTION MOSRALIFE

Invokes the MOSRA *lifetime* computation.

Syntax

```
.OPTION MOSRALIFE=degradation_type_keyword
```

Description

Use this option to invoke device lifetime calculation for the degradation type specified.

The option is used in conjunction with `.OPTION DEGFM="ModelName1=value1 [ModelName2=value2 ...]"`, `.OPTION DegFN=val`, `.OPTION DegFP=val` and `.OPTION DEGF=val`.

The lifetime value is printed in the RADEG file. Lifetime calculation is supported with the Synopsys built-in MOSRA Model Level 1 and Level 3, and with the MOSRA API models.

.OPTION MOSRAPATH

Specifies the path to the MOSRA API `.so` library and allows the automatic platform selection of the correct MOSRA API `.so`.

Syntax

```
.OPTION MOSRAPATH='LIB_DIRECTORY'
```

Description

This option allows the automatic selection of the correct MOSRA API .so library based on the platform on which the simulator is running. This function eliminates the requirement to search manually for the correct platform and allows for efficient MOSRA API .so library distribution and customer applications. The alternative way to specify path to MOSRA API .so is through the PrimeSim HSPICE_MOSRA_MODELS environment variable. For example:

- .OPTION MOSRAPATH='LIB_DIRECTORY' \$ PLATFORM is not needed.
- setenv hspice_mosra_models 'LIB_DIRECTORY/PLATFORM' is optional in the above option and the setenv command.

The simulator uses the following precedence to search for MOSRA API .so:

- mosrapath/
- mosrapath/platform/ \$ platform is appended by the simulator.
- hspice_mosra_models/
- hspice_mosra_models/platform/ \$ platform is appended by the simulator.

.OPTION MOSRASORT

Enables the descending sort for reliability degradation (RADEG) output.

Syntax

```
.OPTION MOSRASORT=degradation_type_keyword
```

Default delvth0

Description

Use this option `mosrasort` to enable the descending sort for reliability degradation (RADEG) output.

If the `mosrasort` option is not specified, or the degradation type keyword is not recognized, MOSRA will not do the sorting. Degradation type keywords are listed in [Table 11](#).

If you only specify the option `mosrasort` and do not specify the degradation type keyword, the simulator sorts RADEG by the `delvth0` keyword.

Examples

In the following usage, MOSRA does the descending sort for RADEG output on `delvth0`'s value.

```
.option mosrasort=delvth0
```

.OPTION MOSRASORTMODE

Option to control whether the degradation sort is device type dependent.

Syntax

```
.OPTION MOSRASORTMODE [= 0|1]
```

Default 0

Description

This option works together with `.OPTION MOSRASORT` to control whether the degradation sorting is irrespective of the device type dependent or not.

If this option does not exist in the netlist, or is explicitly set to 0, the simulator sorts the degradation values for NMOS, PMOS, and MOS Varactor respectively. Prints the sorted NMOS devices first, then PMOS devices, and finally the MOSCAP devices.

If this option is set with no value or to 1, the simulator sorts all output irrespective of the device type and prints in `.radeg` file.

Examples

```
.option mosrasortmode=1
```

.OPTION MRAAPI

Invokes MOSRA simulation using the custom MOSRA API model.

Syntax

```
.OPTION MRAAPI [= 0|1]
```

Default Value if option is not specified in the netlist is 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to load and link the compiled MOSRA API `.so` library file during simulation. If this option parameter is set with no value or to 1, then the MOSRA API `.so` library file is loaded as a dynamically-linked object file.

If this option parameter does not exist in the netlist, or is explicitly set to 0, the MOSRA simulation uses built-in MOSRA model.

Examples

```
.option MRAAPI
```

.OPTION MRADTEMPBA

Device temperature change in RADEG file in MOSRA post stress simulation.

Syntax

```
.OPTION MRADTEMPBA [= 0|1|2]
```

Default Value if option is not specified in the netlist is 1; Value if option name is specified without a corresponding value is 1

Description

Use this option to specify whether device temperature change calculated by MOSRA (dtemp_she) impact post stress simulation. When MRADTEMPBA=0, device temperature change in .radeg (dtemp_she) takes no effect in post stress simulation.

- When MRADTEMPBA=1, both device aging degradations and device temperature change (dtemp_she) in .radeg are back-annotated to device and takes effect in post simulation.
- When MRADTEMPBA=2, only device temperature change is back-annotated in post simulation.

Examples

```
.option MRADTEMPBA
```

.OPTION MRAEXT

Enables access to MOSRA API extension functions.

Syntax

```
.OPTION MRAEXT [= 0|1]
```

Default Value if option is not specified in the netlist is 0; Value if option name is specified without a corresponding value is 1

Description

Use this option to control the access to the MOSRA API extension functions. When MRAEXT=1, the simulator can access the extension functions.

Examples

```
.option MRAEXT
```

.OPTION MRAMCFILE

Controls the generation of the intermediate `RelStep` output files while performing Monte Carlo analysis with MOSRA.

Syntax

```
.OPTION MRAMCFILE=0|1
```

Default 0: Intermediate `RelStep` output files are not generated.

Description

When `MRAMCFILE=1`, generates output files for each `RelStep`.

Examples

```
.option MRAMCFILE=1
```

.OPTION MRAPAGED

Controls the MOSRA API model degradation type.

Syntax

```
.OPTION MRAPAGED=0|1
```

Default 0

Description

If this option is set to 1, then the degradation from the MOSRA API model is the degraded model parameter.

If this option does not exist or is explicitly set to 0, degradations from the MOSRA API model are parameter change with respect to the fresh model, `delta_P`. The simulator calculates the final model parameter by $P + \text{delta_P}$.

Examples

```
.option MRAPAGED=1
```

.OPTION MRA_CONTROL

Sets control option for MOSRA flow.

Syntax

```
.OPTION MRA_CONTROL pdis=value multi_radegfile=value  
custom_agetime=value genmodname=compact|mosra
```

Description

Use this option to set control options for MOSRA flow.

Arguments

Argument	Description
pdis	Enables MOSRA API to access device's PDIS (dissipated power) during transient simulation: 0: disables the access. MRA_VAR->pdis is set to 0. (Default) 1: enables the access. MRA_VAR->pdis is set to device's PDIS.
multi_radegfile	Enables MOSRA to read in multiple input degradation files in post-stress simulation. It works with the .option radegfile control option. 0: reads in only one degradation file in post-stress simulation. The last .option radegfile takes effect when multiple .option radegfile exist. (Default) 1: reads in multiple degradation files in post-stress simulation. All .option radegfile takes effect. Note: This option only supports .MOSRA simmode=1
custom_agetime	Enables MOSRA to read in degradation information from multiple stress time and run post-stress simulation at customized time in the scope of read-in stress time. It works with the .option radegfile control option. 0: disables post-stress run at customized time. (Default) 1: enables post-stress run at customized time. Note: - This option only supports .MOSRA simmode=1. - Customized time is in the scope of read-in stress time.
genmodname	Selects the compact model name or the aging model name passed to the custom aging model, through the variable GENmodName in data structure sMRAModGen. - compact: simulator passes compact model name to aging model. - mosra: simulator passes MOSRA model name to aging model. (Default)

Note:

Traditional options cannot be specified in the .option MRA_CONTROL line.

Examples

Example 16

```
.OPTION MRA_CONTROL pdis=1
```

Example 17

```
.OPTION MRA_CONTROL multi_radegfile=1  
.OPTION RADEGFILE='input1.radeg0'  
.OPTION RADEGFILE='input2.radeg0'
```

Example 18

```
.OPTION MRA_CONTROL multi_radegfile=1 custom_agetime=1  
.OPTION RADEGFILE='5yr.radeg0'  
.OPTION RADEGFILE='15yr.radeg0'  
.MOSRA reltotaltime=10yr simmode=1
```

Example 19

```
.OPTION MRA_CONTROL genmodname=compact
```

.OPTION MRASHETEMP

Option to control whether temperature sent to the MOSRA API model includes temperature change due to self-heating effect.

Syntax

```
.OPTION MRASHETEMP=0|1
```

Default 0

Description

If this option is set to 1 and the compact model self-heating effect is ON, the temperature (variable temp in data structure `smra_var`) MOSRA API model received from the simulator includes the device temperature of the self-heating effect.

If this option is set to 0, the temperature MOSRA API model received does not include the device self-heating effect.

Note:

This option supports the BSIMCMG and UTSOI model.

Examples

```
.option MRASHETEMP=1
```

.OPTION MRA00PATH, MRA01PATH, MRA02PATH, MRA03PATH

These options support user-specified path string access in MOSRA API models.

Syntax

```
.OPTION MRA00PATH = 'file_path1'
.option MRA01PATH = 'file_path2'
.option MRA02PATH = 'file_path3'
.option MRA03PATH = 'file_path4'
```

Default NULL for each path

Description

Use these options to enable global string-type variables such as user-defined paths, for API model developers to access the MOSRA API functions. The user-defined path string is stored in `sMRA_OPT_PATH` data structure in the `sMRA_ENV` structure.

```
typedef struct sMRA_OPT_PATH {
    char *      path0; /* .option MRA00Path (default NULL) */
    char *      path1; /* .option MRA01Path (default NULL) */
    char *      path2; /* .option MRA02Path (default NULL) */
    char *      path3; /* .option MRA03Path (default NULL) */
    /* may be extended, no compatibility issue */
} MRA_OPT_PATH;
```

For the completed `sMRA_ENV` structure, see [MOSRA API Data Structures](#).

.OPTION RADEGFILE

Specifies MOSRA degradation file name for post-stress simulation.

Syntax

```
.OPTION RADEGFILE="file_name"
```

Or

```
.OPTION RADEGFILE="file_name@*"
```

Description

Use this option to specify a MOSRA degradation file name for post-stress simulation. MOSRA read in the degradation information in the specified file, update the device model, and do MOSRA post-stress simulation.

If `.OPTION RADEGFILE="file_name"` is used in `simmode=1` and Monte Carlo analysis, same device degradation is used in all Monte Carlo sweeps of post-stress simulations

Multiple radeg files generated from variation aware stress simulation can be loaded in the variation aware poststress simulation by using `.OPTION RADEGFILE="file_name@*"`. `*` is the wildcard referring to radeg files generated from `simmode=0` and Monte Carlo sweeps. In post-stress simulation with monte carlo, the simulator picks the required radeg file based on the Monte Carlo sweep index.

Examples

Example 20

In this example, the simulator opens file `1.radeg0` and uses the degradation information in this file to perform a post-stress simulation.

```
.mosra reltotaltime='10*365*24*60*60' lin=11 simmode=1
.option radegfile = '1.radeg0'
```

Example 21

In this example, the simulator opens file `1.radeg0` and uses the degradation information in this file for all 10 post-stress simulations by `monte=10`.

```
..mosra reltotaltime='10*365*24*60*60' simmode=1
.option radegfile = '1.radeg0'
.tran 1n 10n sweep monte=10
```

Example 22 `.option radegfile="file_name@*"`

In this example, for each Monte Carlo sweep, the simulator replaces `*` with current Monte Carlo sweep index and opens the corresponding radeg file, for post stress simulation. That is simulator open `1.radeg0@1`, `1.radeg0@2`, `1.radeg0@3`, ... to `1.radeg0@10` for each of the `Monte=10` simulation.

```
.mosra reltotaltime='10*365*24*60*60' simmode=1
.option radegfile = '1.radeg0@*'
.tran 1n 10n sweep monte=10
```

.OPTION RADEGOUTPUT

Outputs the MOSRA degradation information in the `.radeg` file format and/or in a Microsoft Excel CSV format file.

Syntax

```
.OPTION RADEGOUTPUT=YES|NO|CSV|GZ
```

Default YES for non Monte Carlo analysis or Monte Carlo cases with `SimMode` not equal to 2; NO for Monte Carlo analysis with `SimMode=2`.

Description

Use this option to select MOSRA degradation information output file format:

- Generates only the `.radeg` (ASCII format) file.
- Neither the `.radeg` file nor the `.csv` (Microsoft Excel CSV format) file is generated.
- Generates both the `.radeg` file and the `.csv` file.
- Generates only the `.radeg.gz` (Compressed GZ format) file.

Examples

```
.option RADEGOUTPUT=CSV
```

5

MOSRA Output Files

Describes the details of PrimeSim MOSRA output files.

This chapter covers the following topics:

- [MOSRA Output Files](#)

MOSRA Output Files

- [.radeg File](#)
- [.radeg*.csv File](#)
- [Output Files of Post Stress Simulation](#)
- [*.*#@ra.grp File](#)

.radeg File

For each post stress circuit time point, the simulator computes the stress information for each selected transistor and prints the degradation data to the *.radeg file.

```
** $DATA1 SOURCE='HSPICE' VERSION='P-2019.06 64-BIT'
*****Result of Reliability Analysis*****
Circuit time 0.100000E+05      ->Stress time
mn1                          ->Instance name
Device Type: NMOS
SPICE Model Name: nch
L= 1.000000E-07      ->(Instance parameter : l)
W= 5.000000E-06      ->(Instance parameter : w)
M= 1.000000E+00      ->(Instance parameter : m)
NF= 1.000000E+00      ->(Instance parameter : nf)
WEFF= 5.000000E-06      ->(W effect)
Bias Direction: forward
delvth0 = 1.234567E-02      ->(Vth shift)
dids = 90.0000%      ->(ids degradation)
mn2
Device Type: NMOS
SPICE Model Name: nch
```

```
L= 1.000000E-07
W= 5.000000E-06
M= 1.000000E+00
NF= 1.000000E+00
WEFF= 5.000000E-06
Bias Direction: forward
delvth0 = 2.000000E-01
dids = 85.0000%
...
```

.radeeg*.csv File

The simulator can also print the degradation data into a Microsoft Excel CSV format file (.csv file).

```
** $DATA1 SOURCE='HSPICE' VERSION='P-2019.06 64-BIT'
*****Result of Reliability Analysis*****
Circuit time 7.200000E+08
inst,Device Type,SPICE Model Name,L,W,M,NF,WEFF,Bias
Direction,delvth0,dids
mn1,NMOS,nch, 1.000000E-07, 5.000000E-06, 1.000000E+00, 1.000000E+00,
5.000000E-06,forward, 1.234567E-02, 9.000000E-01
mn2,NMOS,nch, 1.000000E-07, 5.000000E-06, 1.000000E+00, 1.000000E+00,
5.000000E-06,forward, 2.000000E-01, 8.500000E-01
...
```

Output Files of Post Stress Simulation

.#@ra=circuit_time_point for TRAN/DC/AC analysis. For the file-naming convention, see the following table:

Analysis Type	Output File Naming Convention
Transient	*.tr#@ra=circuit_time_point
DC	*.sw#@ra=circuit_time_point
AC	*.ac#@ra=circuit_time_point
Transient measurement	*.mt#@ra=circuit_time_point
DC measurement	*.ms#@ra=circuit_time_point
AC measurement	*.ma#@ra=circuit_time_point

***.*#@ra.grp File**

The simulator generates this waveform group file, so that Synopsys WaveView, can load all waveform files of each circuit operation time at once.

6

PrimeSim MOSRA Integration in Simulation Environments

MOSRA analysis is integrated in Synopsys Custom Compiler™ and PrimeWave as well as Cadence® Virtuoso® Analog Design Environment (ADE). Contact the Synopsys support team for these product manual or application notes.

7

Using PrimeSim MOSRA for Aging Analysis

This chapter provides examples of MOSRA in various application scenarios.

This chapter covers the following topics:

- [Basic Analysis](#)
- [Gradual Aging Analysis](#)
- [Process Variation Aware Aging Analysis](#)
- [Self-Heating Aware Aging Analysis](#)
- [MOSRA for Non-Time Domain Transient SPICE Analyses](#)

Basic Analysis

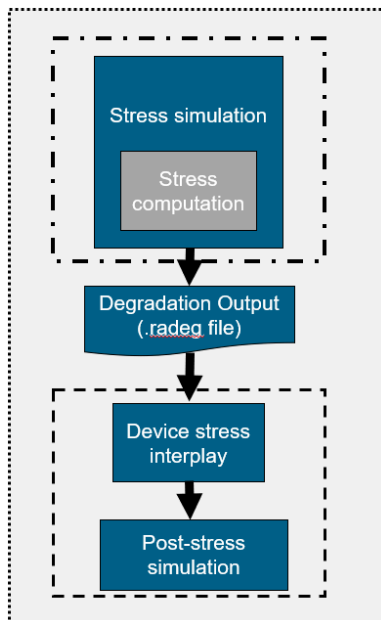
The MOSRA aging simulation flow is described further in the following sections:

- [Stress-Only Simulation](#)
- [Post Stress Only Simulation](#)
- [Post Stress Only Simulation at Customized Time](#)
- [Both Stress and Post Stress Simulation](#)

Stress-Only Simulation

[Figure 3](#) illustrates MOSRA aging simulation flow.

Figure 3 MOSRA Aging Flow



In stress-only simulation, the PrimeSim MOSRA computes the stress for each user-selected MOSFET in the circuit. The stress is computed based on the MOSRA aging model and the biases each device received during the simulation. The stress is integrated over the duration of the transient analysis. The stress integration result is then extrapolated to predict the total stress at the user-specified circuit operation time. Degradation information for each device is printed to the `.radeg` file. The `.radeg` file is a text format file. For more information on this file, see [MOSRA Output Files](#).

Related Commands and Options

```
.MOSRA SimMode=0
```

Example

```
* test circuit using demo MOSRA API model
.temp 27
vdd 1 0 2
mp1 3 2 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn1 3 2 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp2 4 3 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn2 4 3 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp3 2 4 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn3 2 4 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
c1 2 0 .1p
.ic v(2)=2

.model n1 nmos level=54
```

```
+version = 4.4
+vtch0   = 0.25
.model p1 pmos level=54
+version = 4.4
+vtch0   = -0.25

* mos reliability model card, MRA demo models
.model n1_ra mosra level=101
+ rela=1e-4 relb=2 reln=0.25
.model p1_ra mosra level=101
+ rela=1e-4 relb=2 reln=0.25

* appendmodel command
.appendmodel n1_ra mosra n1 nmos
.appendmodel p1_ra mosra p1 pmos

* mosra command, simmode=0 will only run stress simulation
.mosra reltotaltime=1e+4 simmode=0

.tran 1n 100n
.options post accurate
.option mraapi=1
.end
```

Post Stress Only Simulation

In post stress only simulation flow, the degradation information in the `.radeg` file is taken as the input. The simulator reads back the device degradation information, updates the model, and runs the simulation with the aged devices.

Note:

- Circuit time in the `.radeg` file must be consistent with the stress time defined in the netlist.
- Degradation value in `.radeg` file can be edited, different degradation information can be used to see the impact on circuit behavior change.

Related Commands and Options

```
.MOSRA SimMode=1
.option radegfile
```

Example

```
* test circuit using demo MOSRA API model
.temp 27
vdd 1 0 2
mp1 3 2 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn1 3 2 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp2 4 3 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
```

```
mn2 4 3 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp3 2 4 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn3 2 4 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
c1 2 0 .1p
.ic v(2)=2

.model n1 nmos level=54
+version = 4.4
+vth0    = 0.25
.model p1 pmos level=54
+version = 4.4
+vth0    = -0.25

* mos reliability model card, MRA demo models
.model n1_ra mosra level=101
+ rela=1e-4 relb=2 reln=0.25
.model p1_ra mosra level=101
+ rela=1e-4 relb=2 reln=0.25

* appendmodel command
.appendmodel n1_ra mosra n1 nmos
.appendmodel p1_ra mosra p1 pmos

* mosra command, simmode=1 will run post-stress only simulation
* 'simmode0.radeg0' will be taken as input file
.mosra reltotaltime=1e+4 simmode=1
.option radegfile = 'simmode0.radeg0'

.tran 1n 100n
.options post accurate
.option mraapi=1
.end
```

Post Stress Only Simulation at Customized Time

In post stress only simulation at customized time, supporting post-stress time (`ReltotalTime`) defined in `.MOSRA` command inconsistent with Circuit time in the `.radeg` file. The time needs to be in the scope of the Circuit time provided in `.radeg` file, simulator uses degradation information of Circuit time to approximate each device degradation at customized time, then update the model and runs the simulation with aged devices.

Note:

Multiple `relsteps` (`.MOSRA DEC/LIN/LOG/RelStep/RelTimePoints`) are supported.

Related Commands and Options

```
.OPTION MRA_CONTROL custom_agetime
.OPTION radegfile
.MOSRA SimMode=1
```

Example

This example shows post stress only simulation at customized time. Simulator reads in two .radeg files with degradation information at 5yr and 15yr, the defined stress time is 10yr.

```
* test circuit using demo MOSRA API model
.temp 27
vdd 1 0 2
mp1 3 2 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn1 3 2 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp2 4 3 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn2 4 3 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp3 2 4 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn3 2 4 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
c1 2 0 .1p
.ic v(2)=2

.model n1 nmos level=54 +version = 4.4
+vt0 = 0.25
.model p1 pmos level=54 +version = 4.4
+vt0 = -0.25

* mos reliability model card, MRA demo models
.model n1_ra mosra level=101
+ rela=1e-4 relb=2 reln=0.25
.model p1_ra mosra level=101
+ rela=1e-4 relb=2 reln=0.25

* appendmodel command
.appendmodel n1_ra mosra n1 nmos
.appendmodel p1_ra mosra p1 pmos

* mosra command, simmode=1 will run post-stress only simulation at 10yr
* '5yr.radeg0' and '15yr.radeg0' will be taken as input file
* custom_agemode=1 enables post-stress simulation at 10yr
.mosra reltotaltime=10yr simmode=1
.option radegfile = '5yr.radeg0'
.option radegfile = '15yr.radeg0'
.option mra_control multi_radegfile=1 custom_agemode=1

.tran 1n 100n
.options post accurate
.option mraapi=1
.end
```

Both Stress and Post Stress Simulation

.MOSRA simmode=2 invokes both stress and post stress simulation. In this mode, the PrimeSim MOSRA first runs the stress simulation to calculate the device age, then launches the second transient simulation to update devices with the age computed in

the first run and check the circuit behavior change. In `simmode=2`, the circuit ambient condition, stimulus vectors, and the aging model cannot be changed for the stress and post stress runs.

Related Commands and Options

```
.MOSRA SimMode=2
```

Example

```
* test circuit using demo MOSRA API model
.temp 27
vdd 1 0 2
mp1 3 2 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn1 3 2 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp2 4 3 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn2 4 3 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp3 2 4 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn3 2 4 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
c1 2 0 .1p
.ic v(2)=2

.model n1 nmos level=54
+version = 4.4
+vth0    = 0.25
.model p1 pmos level=54
+version = 4.4
+vth0    = -0.25

* mos reliability model card, MRA demo models
.model n1_ra mosra level=101
+ rela=1e-4 relb=2 reln=0.25
.model p1_ra mosra level=101
+ rela=1e-4 relb=2 reln=0.25

* appendmodel command
.appendmodel n1_ra mosra n1 nmos
.appendmodel p1_ra mosra p1 pmos

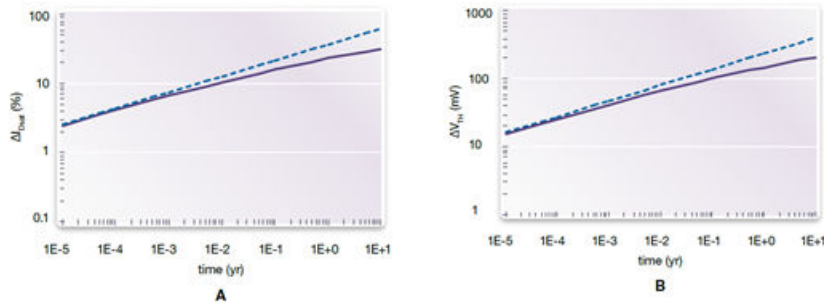
* mosra simmode=2 run both stress and post-stress simulation
.mosra reltotaltime=1e+4 simmode=2

.tran 1n 100n
.options post accurate
.option mraapi=1
.end
```

Gradual Aging Analysis

Due to the interplay of device aging and circuit response to device characteristics changes, gradual aging effect must be accounted for in MOSRA simulation. It is inaccurate to predict a long time aging effect based on one single transient simulation.

Figure 4 The Effect of Accumulated Stress on I_{Dsat} (A) and on V_{TH} (B)



Note:

In Figure 4, the dashed line represents the degradation without the effect of accumulated stress, which produces unnecessary pessimistic circuit lifetime

The PrimeSim MOSRA tool allows you to set up gradual aging simulations based on various mission profiles.

- [Simmode=2 Based Gradual Aging](#)
- [Simmode=3 Based Gradual Aging](#)
- [Simmode=4 Based Gradual Aging — Custom Aging Accumulation](#)

Simmode=2 Based Gradual Aging

MOSRA command `simmode=2` works with MOSRA command arguments of `Relstep` | `DEC` | `LIN` | `LOG` | `Reltimepoints` to achieve gradual aging simulation. MOSRA command arguments of `Relstep` | `DEC` | `LIN` | `LOG` | `Reltimepoints` split the total stress time (`Reltotaltime`) into several segments, and multiple stress and post stress simulations are run on these consecutive stress time intervals. Device models are updated with age effects in the stress simulation of each segment.

In `simmode=2` based gradual aging simulation, multiple stress and post stress simulations are automatically executed by the PrimeSim MOSRA tool. Stress mission profiles, for example, circuit ambient, stress vectors, or aging models are kept unchanged during `simmode=2` based gradual aging simulation.

Related Commands and Options

```
.MOSRA Simmode=2  
.MOSRA Relstep|Reltimepoints|DEC|LIN|LOG
```

Example

```
* test circuit using demo MOSRA API model  
.temp 27  
vdd 1 0 2  
mp1 3 2 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u  
mn1 3 2 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u  
mp2 4 3 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u  
mn2 4 3 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u  
mp3 2 4 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u  
mn3 2 4 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u  
c1 2 0 .1p  
.ic v(2)=2  
  
.model n1 nmos level=54  
+version = 4.4  
+vth0 = 0.25  
.model p1 pmos level=54  
+version = 4.4  
+vth0 = -0.25  
  
* mos reliability model card, MRA demo models  
.model n1_ra mosra level=101  
+ rela=1e-4 relb=2 reln=0.25  
.model p1_ra mosra level=101  
+ rela=1e-4 relb=2 reln=0.25  
  
* appendmodel command  
.appendmodel n1_ra mosra n1 nmos  
.appendmodel p1_ra mosra p1 pmos  
  
* mosra run stress/post-stress simulation for operation time of  
* 2e+3, 4e+3, 6e+3, 8e+3 and 1e+4 respectively.  
.mosra reltotaltime=1e+4 simmode=2 relstep=2e+3  
  
.tran 1n 100n  
.options post accurate  
.option mraapi=1  
.end
```

Simmode=3 Based Gradual Aging

Using `simmode=3` and `.alter netlist` configurations allows you to change aging analysis mission profiles and accumulate the aging effects throughout the circuit operations.

Note:

- If `.option radegfile` is not specified in the top-level netlist, the simulation starts from a fresh device.
- If `.option radegfile` is specified in the top-level netlist, the simulator reads in the last suite of degradations in the radeg file and continues the stress calculation. In consecutive `.alters`, the radeg generated from the previous alter run is read in by simulator.
- The degradation output format is changed when `SimMode=3`. If the MOSRA model has two degradations of the same type, they are output separately to facilitate the continual stress integration and extrapolation.

For example, assume a MOSRA model has two sources of degradation `delvth0`: 10mV due to BTI, and 20mV due to HCI. When `SimMode` is not equal to 3, the two degradations are summed and output as: `delvth0 = 3.0000000e-2` in the radeg file. When `SimMode=3`, the two degradations are output separately: `Delvth0 = 1.000000e-2` and `Delvth0 = 2.000000e-2`.

Related Commands and Options

```
.MOSRA SimMode=3
.option radegfile
```

This is an example of a simulation starting from a fresh device.

From 0 ~ 1e+4 seconds, `vdd = 2V`; for 1e+4 ~ 3e+4 seconds, `vdd=1.99V`; for 3e+4~5e+4 seconds, `vdd=1.98V`.

```
* test circuit using demo MOSRA API model
.temp 27
vdd 1 0 2
mp1 3 2 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn1 3 2 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp2 4 3 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn2 4 3 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp3 2 4 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn3 2 4 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
c1 2 0 .1p
.ic v(2)=2

.model n1 nmos level=54
+version = 4.4
+vth0    = 0.25
.model p1 pmos level=54
+version = 4.4
+vth0    = -0.25

* mos reliability model card, MRA demo models
.model nl_ra mosra level=101
```

```
+ rela=1e-4 relb=2 reln=0.25
.model p1_ra mosra level=101
+ rela=1e-4 relb=2 reln=0.25

* appendmodel command
.appendmodel n1_ra mosra n1 nmos
.appendmodel p1_ra mosra p1 pmos

* Start from fresh device
* Run fresh simulation and report stress degradation at 1e+4 second.
* total stress time 1e+4 second
.mosra reltotaltime=1e+4 simmode=3
.tran 1n 100n
.options post accurate
.option mraapi=1

.alter
vdd 1 0 1.99
* Run post-stress simulation at 1e+4 second. Calculate stress simulation
  for next 2e+4 second. Report total stress degradation at 3e+4 second
* total stress time is 3e+4
* vdd decreases from 2.0v to 1.99v
.mosra reltotaltime=2e+4 simmode=3

.alter
* Run post-stress simulation at 3e+4 second. Calculate stress for another
  2e+4 second. Report total degradation at 5e+4 second
* total stress time is 5e+4, finally run post-stress simulation at 5e+4
  second.
* vdd further decreases from 1.99v to 1.98v
vdd 1 0 1.98
.mosra reltotaltime=2e+4 simmode=3
.end
```

This example shows a simulation starting from an aged device(with a .radeg file as input). It assumes that the degradation information in the .radeg file is for stress time 1e+4 seconds. Output of this simulation is for total stress time of 3e+4 seconds.

```
* test circuit using demo MOSRA API model
.temp 27
vdd 1 0 2
mp1 3 2 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn1 3 2 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp2 4 3 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn2 4 3 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp3 2 4 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn3 2 4 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
c1 2 0 .1p
.ic v(2)=2

.model n1 nmos level=54
+version = 4.4
+vth0    = 0.25
```

```
.model p1 pmos level=54
+version = 4.4
+vth0      = -0.25

* mos reliability model card, MRA demo models
.model n1_ra mosra level=101
+ rela=1e-4 relb=2 reln=0.25
.model p1_ra mosra level=101
+ rela=1e-4 relb=2 reln=0.25

* appendmodel command
.appendmodel n1_ra mosra n1 nmos
.appendmodel p1_ra mosra p1 pmos

* Start from aged device
.mosra reltotaltime=2e+4 simmode=3
.option radegfile='simmode0.radeg0'
.tran 1n 100n
.options post accurate
.option mraapi=1
.end
```

Simmode=4 Based Gradual Aging — Custom Aging Accumulation

In `simmode=2` and `simmode=3` based gradual aging, aging effect accumulation is done by the simulator based on the simulator's built-in algorithm. `Simmode=4` allows custom algorithm for aging accumulation.

In `Simmode=4`, if `.option radegfile` is not specified, the simulation starts as a fresh simulation. If `.option radegfile` is specified, the simulation reads back device degradations from the radeg file, updates device models, and does the stress simulation based on the aged devices. The radeg output of each `simmode=4` run contains the aging degradation of the given stress time interval.

Related Commands and Options

```
.MOSRA SimMode=4
.option radegfile
```

Examples

This example shows a simulation that starts from a fresh device. It has the same effect as `simmode=0`.

```
* test circuit using demo MOSRA API model
.temp 27
vdd 1 0 2
mp1 3 2 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn1 3 2 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp2 4 3 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
```

Chapter 7: Using PrimeSim MOSRA for Aging Analysis

Gradual Aging Analysis

```

mn2 4 3 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp3 2 4 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn3 2 4 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
c1 2 0 .1p
.ic v(2)=2

.model n1 nmos level=54
+version = 4.4
+vth0    = 0.25
.model p1 pmos level=54
+version = 4.4
+vth0    = -0.25

* mos reliability model card, MRA demo models
.model n1_ra mosra level=101
+ rela=1e-4 relb=2 reln=0.25
.model p1_ra mosra level=101
+ rela=1e-4 relb=2 reln=0.25

* appendmodel command
.appendmodel n1_ra mosra n1 nmos
.appendmodel p1_ra mosra p1 pmos

* Start from fresh device
* The simulation report total stress degradation of 1e+4 second
.mosra reltotaltime=1e+4 simmode=4
.tran 1n 100n
.options post accurate
.option mraapi=1
.end

```

This example shows a simulation that starts from an aged device (with `.radeg` file as input). It assumes that the degradation information in the `.radeg` file is for stress time `1e+4` seconds, different from `simmode=3`, this simulation reports degradation introduced by the current `reltotaltime` of `2e+4` seconds.

```

* test circuit using demo MOSRA API model
.temp 27
vdd 1 0 2
mp1 3 2 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn1 3 2 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp2 4 3 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn2 4 3 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp3 2 4 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn3 2 4 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
c1 2 0 .1p
.ic v(2)=2

.model n1 nmos level=54
+version = 4.4
+vth0    = 0.25
.model p1 pmos level=54

```

```
+version = 4.4
+vth0     = -0.25

* mos reliability model card, MRA demo models
.model nl_ra mosra level=101
+ rela=1e-4 relb=2 reln=0.25
.model pl_ra mosra level=101
+ rela=1e-4 relb=2 reln=0.25

* appendmodel command
.appendmodel nl_ra mosra nl nmos
.appendmodel pl_ra mosra pl pmos

* Start from aged device
* this simulation report degradation of 2e+4 second, the current stress
  time interval.
.mosra reltotaltime=2e+4 simmode=4
.option radegfile='simmode0.radeg0'
.tran 1n 100n
.options post accurate
.option mraapi=1
.end
```

Outputs for this example are: `simmode4.tr0@ra=10000` (post-stress simulation for 1e+4 seconds) and `simmode4.radeg0` (degradation result for the current 2e+4 seconds stress time interval).

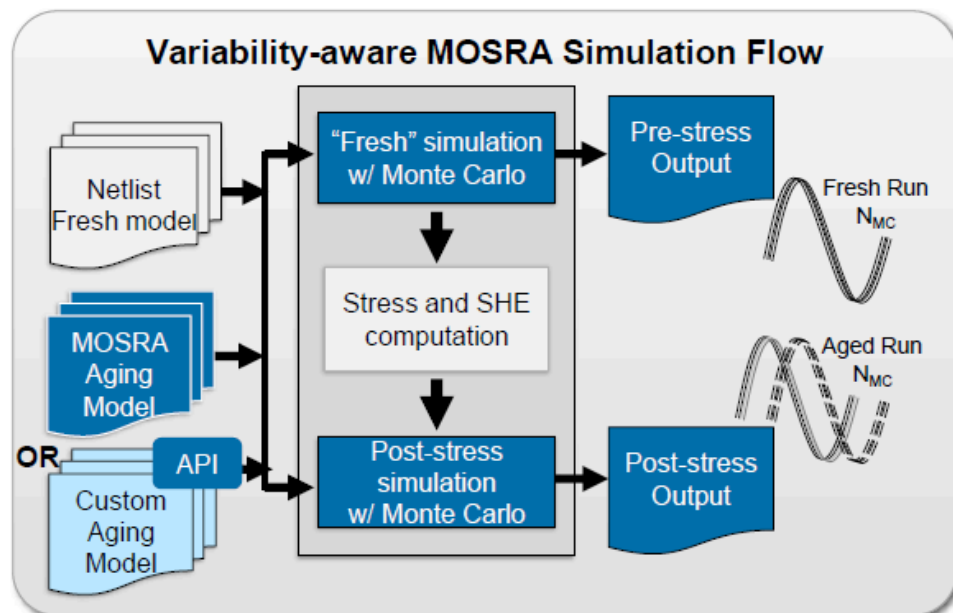
You can use `simmode0.radeg0` and `simmode4.radeg0` to calculate the accumulated aging effect with your own algorithm.

Process Variation Aware Aging Analysis

With process technology scaling down, semiconductor fabrication induced process variation and its impact on design performance and yield must be accounted through variability-aware design flow. Further, to accurately capture the EOL statistical behavior

by considering the interplay of process variation and reliability aging variation, variability-aware reliability analysis is required with the flow as shown in [Figure 5](#):

Figure 5 Variability Aware MOSRA Simulation Flow



The PrimeSim MOSRA tool supports several flows for variability aware aging analysis, including variability aware stress-only analysis, variability aware post stress-only analysis, and variation aware both stress and post stress analysis.

Variability aware stress-only analysis uses `simmode=0` and `monte sweep=N`. There are N simulations conducted.

Variability aware post stress-only analysis uses `simmode=1` and `monte sweep=N`. In this flow:

- If you specify `.OPTION RADEGFILE="file_name"`, where "`file_name`" points to one `radeg` file, then all Monte Carlo sweeps of `simmode=1` post stress simulations use the same device degradations from the specified one `radeg` file.
- If you use `.OPTION RADEGFILE="file_name@*"` to point to a group of `radeg` files generated by Monte Carlo simulation with `SIMMODE=0` or `SIMMODE=2`, each MOSRA `simmode=1` post stress simulation reads back the corresponding `radeg` file based on the Monte Carlo sweep index. You can also selectively run a specific Monte Carlo sweep index and `simmode=1` post stress simulation.

Variability aware MOSRA for both stress and post stress analysis uses `simmode=2` and `monte sweep=N`. In this flow, a total of $2N$ simulations are required.

Examples

This example shows that the same degradation information is read back from `simmode0.radeg0` for each of the 10 monte sweep runs of `simmode=1` post stress simulation.

```
* test circuit using demo MOSRA API model
.temp 27
.option macmod=3
.param random1=agauss(0,1,1)
.param devth0 = 'random1*0.1'
vdd 1 0 2
mp1 3 2 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn1 3 2 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp2 4 3 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn2 4 3 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp3 2 4 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn3 2 4 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
c1 2 0 .1p
.ic v(2)=2

.model n1 nmos level=54
+version = 4.4
+vtth0    = 0.25+devth0
.model p1 pmos level=54
+version = 4.4
+vtth0    = -0.25-devth0

* mos reliability model card, MRA demo models
.model n1_ra mosra level=101
+ rela=1e-4 relb=2 reln=0.25
.model p1_ra mosra level=101
+ rela=1e-4 relb=2 reln=0.25

* appendmodel command
.appendmodel n1_ra mosra n1 nmos
.appendmodel p1_ra mosra p1 pmos

* Start from aged device
.mosra reltotaltime=1e+4 simmode=1
.option radegfile='simmode0.radeg0'
.tran 1n 100n sweep monte=10
.options post accurate
.option mraapi=1
.end
```

In this example, a total of 10 radeg files are generated from previous `simmode=0` with sweep monte=10 simulations; then, the `simmode=1` simulation with sweep monte=10 reads back each corresponding radeg file.

```
* test circuit using demo MOSRA API model
.temp 27
```

Chapter 7: Using PrimeSim MOSRA for Aging Analysis

Process Variation Aware Aging Analysis

```
.option macmod=3
.param random1=agauss(0,1,1)
.param devth0 = 'random1*0.1'
vdd 1 0 2
mp1 3 2 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn1 3 2 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp2 4 3 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn2 4 3 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp3 2 4 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn3 2 4 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
c1 2 0 .1p
.ic v(2)=2

.model n1 nmos level=54
+version = 4.4
+vtth0    = 0.25+devth0
.model p1 pmos level=54
+version = 4.4
+vtth0    = -0.25-devth0
* mos reliability model card, MRA demo models
.model n1_ra mosra level=101
+ rela=1e-4 relb=2 reln=0.25
.model p1_ra mosra level=101
+ rela=1e-4 relb=2 reln=0.25

* appendmodel command
.appendmodel n1_ra mosra n1 nmos
.appendmodel p1_ra mosra p1 pmos

* Start from aged device
.mosra reltotaltime=1e+4 simmode=1
.option radegfile='simmode0_mc.radeg0@*'
.tran 1n 100n sweep monte=10
.options post accurate
.option mraapi=1
.end
```

In this example, a total of 10 radeg files are generated from previous `simmode=0` with `sweep monte=10` simulations; the `simmode=1` with Monte sweep list of 3-7 reads back `simmode0_mc.radeg0@3-simmode0_mc.radeg0@7` respectively.

```
* test circuit using demo MOSRA API model
.temp 27
.option macmod=3
.param random1=agauss(0,1,1)
.param devth0 = 'random1*0.1'
vdd 1 0 2
mp1 3 2 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn1 3 2 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp2 4 3 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn2 4 3 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp3 2 4 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn3 2 4 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
```

```
c1 2 0 .1p
.ic v(2)=2

.model n1 nmos level=54
+version = 4.4
+vth0     = 0.25+devth0
.model p1 pmos level=54
+version = 4.4
+vth0     = -0.25-devth0

* mos reliability model card, MRA demo models
.model n1_ra mosra level=101
+ rela=1e-4 relb=2 reln=0.25
.model p1_ra mosra level=101
+ rela=1e-4 relb=2 reln=0.25

* appendmodel command
.appendmodel n1_ra mosra n1 nmos
.appendmodel p1_ra mosra p1 pmos

* Start from aged device
.mosra reltotaltime=1e+4 simmode=1
.option radegfile='simmode0_mc.radeg0@*'
.tran 1n 100n sweep monte=list(3:7)
.options post accurate
.option mraapi=1
.end
```

Self-Heating Aware Aging Analysis

The device self-heating effect (SHE) in transistors from advanced process technologies like FinFET and FDSOI must be considered in the aging simulation, because the increased temperature enhances the aging effect. The PrimeSim MOSRA tool enables SHE aware aging simulation through `STRESS_DTEMP` and `.OPTION MRADTEMPBA`, where `STRESS_DTEMP` is a special MOSRA degradation type for you to compute device SHE effect in the MOSRA API model, and `option MRADTEMPBA` controls the temperature change in the radeg file read-back for post age simulation.

As shown in [Figure 5](#), the device SHE and its impact to aging is accounted in the MOSRA flow and no additional simulation step is required.

Note:

You can use `STRESS_DTEMP` to estimate device self-heat effect based on average power the device consumes, and incorporate the self-heat effect into the aging model. For details, contact Synopsys technical support.

MOSRA for Non-Time Domain Transient SPICE Analyses

Device aging calculation requires time-domain transient analysis. Therefore, MOSRA stress simulation must use `.tran` analysis. However, MOSRA `simmode=1` can be used with any SPICE analyses such as DC, AC, Noise, HB, SN, and so on, to characterize the change in the circuit figures of merits after the devices are aged. A previously-generated device degradation file, the `radeg` file, based on user-defined stress condition must be loaded through the `.option radegfile`.

The syntax is as follows:

```
.mosra reltotaltime='10*365*24*60*60' lin=11 simmode=1
.option radegfile = 'file name'
```

Related Commands and Options

```
.MOSRA SimMode=1
.option radegfile
```

Example

The following example netlist performs DC and AC simulation in MOSRA post-stress:

```
* MOSRA DC/AC TEST
.option radegfile = 'dc_simmode0.radeg0'
vdd 1 0 -2 ac=1
mp1 1 2 0 0 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
vgs 2 0 -2

.model p1 pmos level=54 version=4.5
.model p1_ra mosra level=101
+ rela=1e-4 relb=2 reln=0.25

.appendmodel p1_ra mosra p1 pmos
.mosra reltotaltime='10*365*24*60*60' lin=11 simmode=1
.dc vdd 0 -1.2 -0.1
.ac dec 1 1e5 1e9
.options post mraapi=1
.print dc i(vdd)
.print ac i(vdd) ii(vdd)
.end
```

The simulator parses the user-specified `radeg` file, conducts the post-stress simulation in the specified `RelStep` (stress time), and generates the following DC output files:

- `dcac.sw0@ra=3.1536e+07`
- `dcac.sw0@ra=6.3072e+07`

- `dcac.sw0@ra=9.4608e+07`
- `dcac.sw0@ra=1.26144e+08`
- `dcac.sw0@ra=1.5768e+08`
- `dcac.sw0@ra=1.89216e+08`
- `dcac.sw0@ra=2.20752e+08`
- `dcac.sw0@ra=2.52288e+08`
- `dcac.sw0@ra=2.83824e+08`
- `dcac.sw0@ra=3.1536e+08`
- `dcac.ac0@ra=3.1536e+07`
- `dcac.ac0@ra=6.3072e+07`
- `dcac.ac0@ra=9.4608e+07`
- `dcac.ac0@ra=1.26144e+08`
- `dcac.ac0@ra=1.5768e+08`
- `dcac.ac0@ra=1.89216e+08`
- `dcac.ac0@ra=2.20752e+08`
- `dcac.ac0@ra=2.52288e+08`
- `dcac.ac0@ra=2.83824e+08`
- `dcac.ac0@ra=3.1536e+08`

For detailed information on the post stress simulation output files, see [MOSRA Output Files](#).

8

MOSRA API for Custom Aging Model Development

This section describes how to use the MOSRA API to add a custom reliability model.

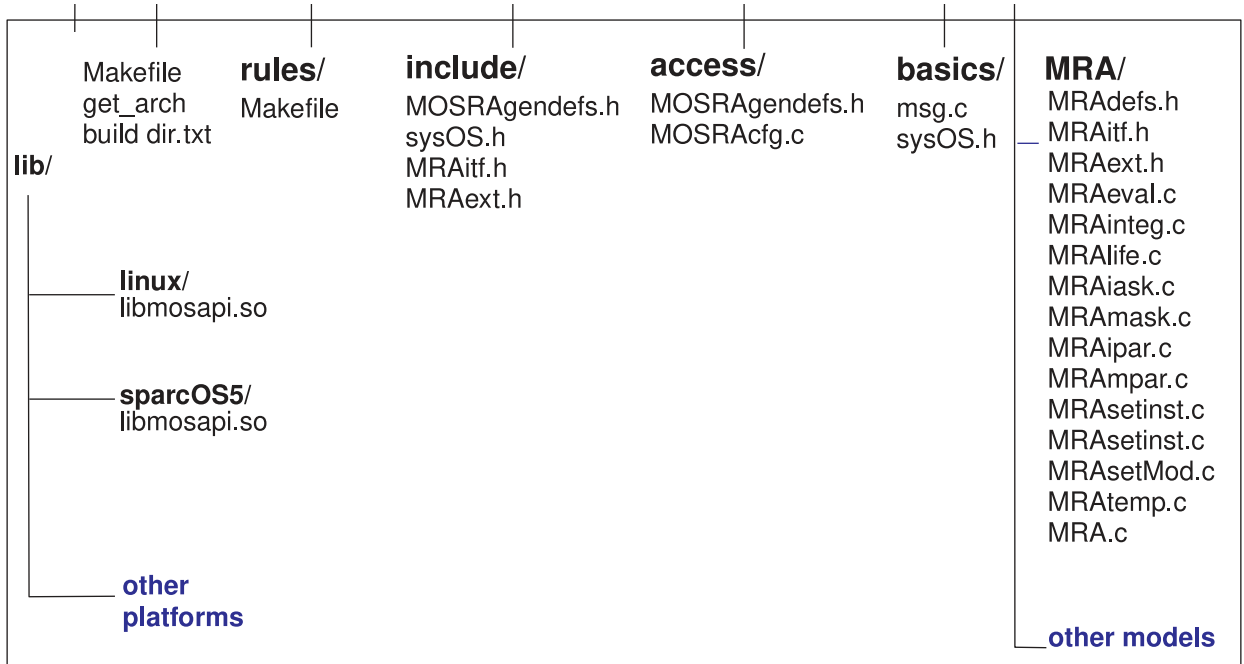
This chapter covers the following topics:

- [MOSRA API Example Package and MOSRA API Functions and Data Structures](#)
- [Integration-Based Aging Calculation Using MOSRA API](#)
- [Adding Custom Reliability Models](#)
- [Other Functions Supported in MOSRA API](#)
- [MOSRA API Supported Aging Degradations](#)
- [MOSRA API Data Structures](#)
- [MOSRA API Model Function Prototypes](#)

MOSRA API Example Package and MOSRA API Functions and Data Structures

The MOSRA API package provides a base directory, `./MOSRAAPI`, which contains files and sub directories listed in [Figure 6](#).

Figure 6 MOSRA API Directory Structure



Data Structures in include/MOSRAgendef.h File

Table 4 Data Structures in MOSRAgendef.h File

Structure/Enum Type	Definition
typedef enum <i>StressType_E</i>	This enum type defines the degradation types that the MOSRA API supports.
typedef struct <i>sMRAmoGen</i>	This structure defines the general structure of MOSRA model parameters.
typedef struct <i>sMRAinstGen</i>	This structure defines the general structure of MOSRA instance parameters.
typedef struct <i>sMRAstress</i>	This structure defines the general structure of a MOSRA stress.
typedef struct <i>sMRAparm</i>	This structure defines the MOSRA parameter's id according to the parameter's keyword.
typedef struct <i>sMRA_ENV</i>	This structure defines the environment parameters. See MOSRA API Data Structures for more details.
typedef enum <i>MOSRAstatus_E</i>	This enum type specifies the MOSRA function return status.
typedef struct <i>sMRApublic</i>	This structure contains the information available to the simulator about a particular MOSRA model. The simulator presents the frontend with an array of pointers to these structures which it uses to determine legal MOSRA model types and parameters.
typedef struct <i>sMRAgeneric</i>	This structure defines the generic type of a particular MOSRA model and contains the information available to the simulator about the MOSRA model.

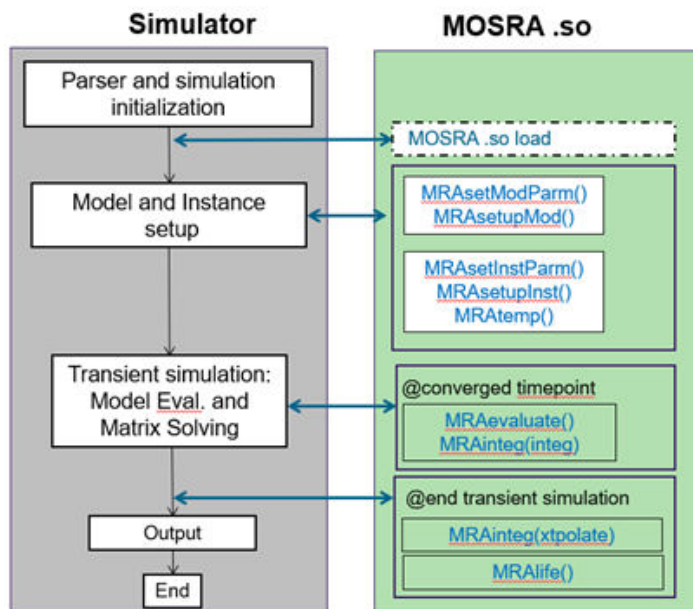
MOSRA API Function Prototypes in include/MRAext.h File

```
extern INT32_t MRASetaModParam();
extern INT32_t MRAsetaModParam();
extern INT32_t MRAsetaInstParam();
extern INT32_t MRAsetaInstParam();
extern INT32_t MRAsetaupModel();
extern INT32_t MRAsetaupInst();
extern INT32_t MRAtemp();
extern INT32_t MRAevaluate();
extern INT32_t MRAinteg();
extern INT32_t MRAlife();
```

For detailed information on these function arguments, see [MOSRA API Model Function Prototypes on page 106](#).

These functions are called by the simulator in the following sequence:

Figure 7 Interoperation of Simulator and MOSRA API



Public Interface Functions in access/MOSRAcfg.c File

Table 5 Public interface functions in MOSRAcfg.c file

Definition	Definition
<code>#define MOSRASIZE 1</code>	MOSRASIZE is the total number of MOSRA models included in the MOSRA API.
<code>void getMRAModTblSize(int *size)</code> Argument: <i>size</i> output (int *): The total number of the MOSRA models which are included in the MOSRA API	This interface function returns the number of MOSRA models included in the library.
<code>MRAgeneric_t *getMRAMod(int idx)</code> Argument: <i>idx</i> input (int): The index of the MOSRA model included in the MOSRA API.	This interface function returns the address of the MOSRA model general type, according to the index of the MOSRA model included in the MOSRA API.
<code>void getMRAstart(MRA_ENV *ckt)</code> Argument: <i>ckt</i> input (struct MRA_ENV *): Pointer to the circuit environment parameter structure.	This interface function informs MOSRA simulation start, so that MOSRA model can conduct initialization tasks. For example, it opens a file, allocates global memory, and initializes global variable.
<code>void getMRAend(MRA_ENV *ckt)</code> Argument: <i>ckt</i> input (struct MRA_ENV *): Pointer to the circuit environment parameter structure.	This interface function informs MOSRA simulation end. Use this function to close files opened in MOSRA, free global memory, and reset global variable.
<code>MOSRApostFlag_t MRApostExtrapolation(MRA_ENV *ckt)</code> Argument: <i>ckt</i> input (struct MRA_ENV *): Pointer to the circuit environment parameter structure.	This interface function informs MOSRA extrapolation for all devices end. Actions that needs to be done after extrapolation can be implemented in this function. It returns the status defined in MOSRAgends.h.

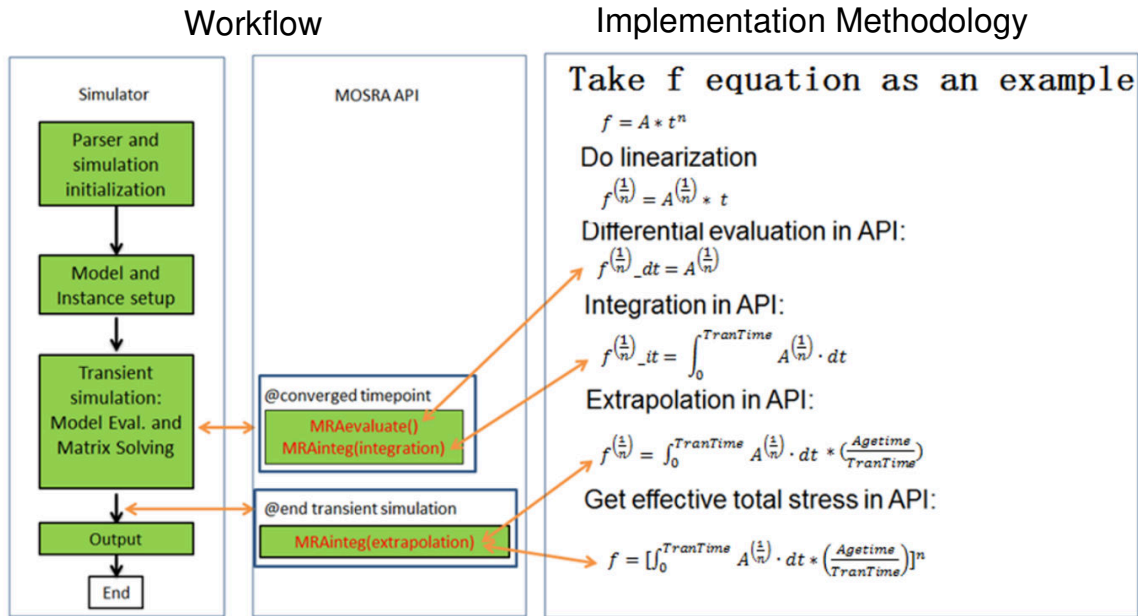
Integration-Based Aging Calculation Using MOSRA API

In general, the aging equation follows the following formulation:

$$Age = f(V, I, \dots) \times t^n$$

Figure 8 shows the methodology to implement such an equation in the MOSRA API. To simplify the equation, $f(V, I, \dots)$ is taken as A in the example.

Figure 8 Implementation Methodology Workflow



Note:

The code snippet in Figure 8 is based on the formulation of $f = A * t^n$. For custom aging formulation with complicated time-dependency, special attention is required when deploying the model using the MOSRA API, so as to ensure the model working properly in gradual aging scenario. For any assistance, contact Synopsys technical support team.

Adding Custom Reliability Models

This section describes the steps to add a custom reliability model to the MOSRA API. A sample custom reliability model is built into the ./MRA directory under the ./MOSRAAPI directory. You can start with this sample model to make it your own.

- [Introduction to the Files Under MRA](#)
- [Adding a New MOSRA Model Parameter](#)
- [Defining the MOSRA Model Level and Other Generic Types](#)
- [Custom Aging Formulation Deployment](#)
- [How to Create a New MOSRA Model Directory](#)
- [Supported Platforms](#)

- [Compilation Environment Setup](#)
- [MOSRA Library Quality](#)
- [Setting Up the Runtime Shared-Library Path](#)
- [Netlist Setup](#)

Introduction to the Files Under MRA

Table 6 Files in ./MOSRAAPI/MRA Directory

File	Description
MRAdefs.h	User header file structure definitions for the model MOSRA model.
MRAitf.h	User header file for the MOSRA generic type.
MRAext.h	User header file declares the MOSRA model prototypes.
MRAiask.c	Includes the function MRAgetInstParam; this function is used to return the instance parameter's value.
MRAmask.c	Includes the function MRAgetModParam; this function is used to return the model parameter's value.
MRAeval.c	Includes the function MRAevaluate, where you define your reliability model equations.
MRAipar.c	Includes the function MRAsetInstParam.
MRAmpar.c	Includes the function MRAsetModParam.
MRAtemp.c	Includes the function MRAtemperature.
MRA.c	Includes the model and instance's parameter names.
MRAsetInst.c	Includes the function MRAsetupInst.
MRAsetMod.c	Includes the function MRAsetupModel.
MRAinteg.c	Includes the function MRAintegration.
MRAlife.c	Includes the function MRAlifetimeDeg.

MRA Model Data Structures MOSRAAPI/MRA/MRAdefs.h

Instance and model parameters also are defined in this structure:

Table 7 MRA model data structures in MRAdefs.h file

Structure	Definition
<code>typedef struct sMRAModel</code>	This structure defines MRA model-related information such as MRAModName, MRAModType, and model parameters.
<code>typedef struct sMRAInstance</code>	This structure defines MRA instance-related information such as MRAName and arrays of MRAstress_t to store the stress value, type, and n coefficient.
Instance and model parameters are also defined in this file, for example <code>#define MRA_MOD_RELA 101#define MRA_MOD_RELB 102 ...</code>	These structures define the instance and model parameters' id.

Model Data Structure in MOSRA API: MOSRAAPI/MRA/MRAitf.h

Table 8 MRA model data structure in MRAitf.h file

Function	Definition
<code>#define MRA_LEVEL 101</code>	MRA_LEVEL is the MRA model's level number.
<code>extern MRAparam_t MRAipTable[];</code>	Table of instance parameters.
<code>extern MRAparam_t MRAmPTable[];</code>	Table of model parameters.
<code>extern INT32_t MRAipTSize;</code>	Parameter number of table of instance parameters.
<code>extern INT32_t MRAmPTSize;</code>	Parameter number of table of model parameters.
<code>extern INT32_t MRAiSize;</code>	Size of instance structure.
<code>extern INT32_t MRAmSize;</code>	Size of model structure.
<code>MRAgeneric_t MRAdef</code>	The generic type of MRAdef contains all the interface information available to the simulator about the MRA model and the addresses of all the function prototypes of the MRA model.

MRA Parameter Keyword Tables in the MOSRAAPI/MRA/MRA.c File

Table 9 MRA parameter keyword tables in MRA.c file

Parameter Keyword	Definition
<code>MRAparam_t MRAipTable[]</code>	This table maps the MRA instance parameter keyword with the instance parameter id.

Table 9 MRA parameter keyword tables in MRA.c file (Continued)

Parameter Keyword	Definition
<code>MRAparm_t MRAmPTable[]</code>	This table maps the MRA model parameters keyword with the model parameter id.
<code>INT32_t MRAipTSize = NUMELEMS(MRAipTable);</code> <code>INT32_t MRAmPTSize = NUMELEMS(MRAmPTable);</code> <code>INT32_t MRAmSize = sizeof(MRAmodel_t);</code> <code>INT32_t MRAiSize = sizeof(MRAinstance_t);</code>	These keywords define the size of instance parameter table, model parameter table, MRA model, and instance structure.

Adding a New MOSRA Model Parameter

You can add a new MOSRA model parameter to the API which may be specified in the `.MODEL` statement in your SPICE model card.

For example, to add a model parameter named `mra_param1`:

1. Modify the `MRAdefs.h` file:

Inside `MRAdefs.h`, there are two structures:

- “`MRAmodel_t`” is the model structure which includes the model parameters.
- “`MRAinstance_t`” is the instance structure which includes the instance stress and parameters.

Add the following two lines in the `MRAmodel_t` structure:

```
double mra_param1;
unsigned mra_param1Given :1;
```

Add the following line at the end of `MRAdefs.h`

```
#define MRA_MOD_MRA_PARAM1 169
```

2. Modify `MRA.c`

Add the following line in the `MRAparm_t MRAmPTable[ ]`

```
PARAM("mra_param1",MRA_MOD_MRA_PARAM1) ,
```

3. Modify `MRAmpar.c`

Before the lines:

```
default:
return(MOSRA_E_BADPARAM);
```

Add these lines:

```
case MRA_MOD_MRA_PARAM1 :  
model->mra_param1 = value;  
model->mra_param1Given = TRUE;  
break;
```

Defining the MOSRA Model Level and Other Generic Types

You can define the level in `MRAith.h` with the keyword `MRA_LEVEL`.

Note:

You must set all user-defined reliability model levels as a 3-digit number, starting from 100.

Here is an example of the `MRAitf.h` file:

```
#ifndef __MRA_ITF_H_  
#define __MRA_ITF_H_  
  
#include "MOSRAgendefs.h"  
#include "MRAext.h"  
  
#define MRA_LEVEL 101    /*defines the level of this mosra model */  
  
/* instance parameters table */  
extern MRAparam_t      MRAipTable[ ];  
/* model parameters table */  
extern MRAparam_t      MRAmPTable[ ];  
/* size of instance parameters */  
extern INT32_t          MRAipTSize;  
/* size of model parameters */  
extern INT32_t          MRAmPTSize;  
/* size of instance structure */  
extern INT32_t          MRAiSize;  
/* size of model structure */  
extern INT32_t          MRAmSize;  
  
MRAGeneric_t MRAdef = {  
{ "MOSRA3.0",  
  "customer dummy MOSRA model level=101",  
  /* Don't forget to change the information message to YOUR Level number */  
  
  MRA_LEVEL,  
  &MRAmPTSize,  
  MRAmPTable,  
  &MRAipTSize,  
  MRAipTable,  
  &MRAmSize,  
  &MRAiSize
```

```

    },

    &MRAgetModParam, /* match the model parameter's name and return this
        parameter's value */
    &MRAsetModParam, /* set the model parameter's value */
    &MRAgetInstParam, /* match the instance parameter's name and return this
        parameter's value */
    &MRAsetInstParam, /* set the instance parameter's value */
    &MRAsetupModel, /* Setup model parameters default value and
        check */
    &MRAsetupInst, /* Setup instance parameters default value
        and check */
    &MRAtemperature, /* MOSRA binning and temperature function */
    &MRAevaluate, /* MOSRA model equation */
    &MRAinteg, /* MOSRA integration and extrapolation function */
    &MRAlife /* MOSRA lifetime function */
};
#endif /* __MRA_ITF_H__ */

```

Custom Aging Formulation Deployment

Use the `*eval.c` file to define your own reliability model equations (using the demo directory as an example, it is the `MRAeval.c` file). The following two examples show the implementation of reliability equations in the `MRAeval.c` file.

Defining Threshold Voltage Degradation

This example implements a threshold voltage degradation equation, $\Delta V_{th} = a \times V_{gs}^b \times t^n$ in the `MRAeval.c` file, where a , b , and n are reliability model parameters and V_{gs} is the stress between gate and source.

The following example illustrates the flow of how the example code works.

```

{
    MRAModel_t *model = (MRAModel_t *)mod;
    MRAinstance_t *here = (MRAinstance_t *)inst;

    INT32_t type, i;
    double deltavth=0;
    type = model->MRAModType;

    /* dummy stress equation */
    stress1 = model->MRAa * pow(inOut->vgs, model->MRAb);
    /* linearization */
    Stress_dt = pow(stress1, 1/model->MRAreln);
    /* Initialization of the stress. Don't modify this part. */
    for (i = 0; i < 5; i++) {
        here->stress[i].given = 0;
        here->stress[i].value = 0.0;
        here->stress[i].type = STRESS_TOTALSIZE;
        here->stress[i].n = 0.0;
    }
}

```

```

}

/* stress value */
here->stress[0].type = STRESS_VTH;    // stress type
here->stress[0].value = Stress_dt;    // linearization value
here->stress[0].n = model->MRAreln;  // This is the real
    "n"here->stress[0].given = TRUE;

```

The stress equation could be an expression of model parameters and the *inOut* elements, variables in *inOut* data structure include:

```

double tnom; /* model norm temperature, in degree-K */
double temp; /* simulation temperature, in degree-K */
double vds; /* processed vds bias */
double vgs; /* processed vgs bias */
double vbs; /* processed vbs bias */
double w; /* transistor width in meter unit */
double l; /* transistor length in meter unit */
double leff; /* effective channel length in meter unit */
double weff; /* effective channel width in meter unit */
double vth; /* threshold voltage (i.e., turn-on voltage) */
double ids; /* drain (channel) current (not include isub) */
double gds; /* output conductance (dIds/dVds) */
double gm; /* trans-conductance (dIds/dVgs) */
double isub; /* impact ionization current */
double tox_ra; /* mosfet oxide */
double vd; /* mosfet internal drain voltage */
double vg; /* mosfet internal gate voltage */
double vs; /* mosfet internal source voltage */
double vb; /* mosfet internal bulk voltage */
double vd_ext; /* mosfet external drain voltage */
double vg_ext; /* mosfet external gate voltage */
double vs_ext; /* mosfet external source voltage */
double vb_ext; /* mosfet external bulk voltage */
...
double ibulk; /* bulk current */
double v_node5; /* voltage on internal node 5 */
double v_node5_ext; /* voltage on external node 5 */

```

Defining Ids Degradation

This example implements an Ids degradation equation $\Delta I_{ds} = 0.1 \cdot a V_{gs}^b t^n$ in the [MRAeval.c](#) file; where *a*, *b*, and *n* are reliability model parameters and V_{gs} is the stress between gate and source. A similar formulation adds Ids degradation in the [MRAeval.c](#) file:

```

...
INT32_t type, i;
double deltavth=0;
double deltaIds = 0;    // Define deltalds
...
/* added equation here */

```

```
deltavth = model->MRArela * pow(inOut->vgs, model->MRArelb);

/* */
/* Ids degradation equation */
deltaIds = model->MRArela * pow(inOut->vgs, model->MRArelb) * 0.1;
deltaIds = pow(deltaIds, 1/model->MRAreln);
...
/* output the stress */
here->stress[0].type = STRESS_VTH;
here->stress[0].value = deltavth;
here->stress[0].n = model->MRAreln;
here->stress[0].given = TRUE;
/* Ids degradation */
here->stress[1].type = STRESS_DIDS;
here->stress[1].value = deltaIds;
here->stress[1].n = model->MRAreln;
here->stress[1].given = TRUE;

return (MOSRA_OK);
}
```

How to Create a New MOSRA Model Directory

If you have finished the aging formulation implementation but want to build a new model which uses different file names, model parameters, and model level, you must execute the tasks as described in the following sections:

- [Creating a New MOSRA Model Directory](#)
- [Modifying the MOSRAcfg.c File Under the ./access Directory](#)

Creating a New MOSRA Model Directory

To create a new MOSRA model directory:

1. Copy the demo model directory ./MRA to a new directory. For example:

```
> cp -R ./MRA ./DDD
```

2. Change file prefix for files in the new directory from MRA to the new model directory name, such as “DDD”. For example, the MRA.c file becomes DDD.c, and MRAext.h becomes DDDext.h.

3. Replace all “MRA” with “DDD” in all *.h and *.c files, but reserve the following eight words:

“MRAinstGen”, “MRAmodGen”, “MRA_ENV”, “MRA_VAR”, “MRAparm_t”,
“MRAgeneric_t”, “MRAstress_t”, and “MRApublic_t” because they are defined in
access/MOSRAgends.h as generic definitions for all MOSRA models.

After making the replacements noted above, the `MRA.c` file becomes the `DDD.c` file in the `./DDD` directory, with `MRAParam_t` remaining the same; it now looks like:

```
#include "sysOS.h"
#include "MOSRAgends.h"
#include "DDDdefs.h"

MRAParam_t DDDipTable[] = {
    PARAM("xxx", DDD_IGNORED),
};

MRAParam_t DDDmPTable[] = {
    PARAM("rela", DDD_MOD_RELA),
    PARAM("relb", DDD_MOD_RELB),
    PARAM("reln", DDD_MOD_RELN),

    /* added new model parameter there */
};

INT32_t DDDipTSize = NUMELEMS(DDDipTable);
INT32_t DDDmPSize = NUMELEMS(DDDmPTable);
INT32_t DDDmSize = sizeof(DDDmodel_t);
INT32_t DDDiSize = sizeof(DDDinstance_t);
```

Modifying the `MOSRAcfg.c` File Under the `./access` Directory

To update your configuration file:

1. Add your include file. For example:

```
#include "DDDitf.h"
```

2. Add the `switch` statement, following the “MRA” example. For example:

```
...
switch (idx) {
    case 0:
        return (&MRAdef); /* the MRAdef is the MRA model
        generic structure defined in the MRAitf.h */
    case 1:
        return (&DDDdef); /* the DDDdef is the DDD model generic
        structure which is defined in the DDDitf.h */
    ...
}
```

3. Increase the MOSRA Size Number by 1. `MOSRASIZE` indicates the number of different models included in the API. For example:

```
#define MOSRASIZE 2
```

There are now two MOSRA models defined in this API: MRA and DDD

4. Continue creating your MOSRA model following steps described in [Creating a New MOSRA Model Directory on page 85](#).

Supported Platforms

The MOSRA API supports the platforms listed in [Table 10](#).

Table 10 Supported Platforms for MOSRA API

Platform	Operating System	Synopsys Platform Keyword
IA-64	RHEL	linux64
Intel EM64T	SUSE Enterprise Linux	suse64
PC	Windows	win64

Compilation Environment Setup

If you have added the compiler path to your PATH environment variable, skip this procedure.

The recommended compiler is gcc 4.8.3 or above on Linux. If you cannot find a compiler by using the command `which gcc`, then you should either add the path to your setting or use the command-line `make` which is described in the following section by using a custom compiler and options.

Library Compilation

The following are the supported compilation commands on Linux use them under `./mosraapi`:

- `make` for 64-bit release version build
- `make DEBUG=1` for 64-bit debug version build
- `make clean` clean up the `obj/lib/` directory for specific platform files
- `make cleanall` delete the `obj/lib/` directories directly
- `make CC="your_compiler_path" CCFLAGS="your_compiling_options" LDFLAGS="your_link_options"` to use custom compiler and options

Following are the supported compilation commands on Windows:

- `make win=1` for 64-bit release version build
- `make win=1 DEBUG=1` for 64-bit debug version build
- `make win=1 clean` clean up the `obj/lib/` directory for specific platform files
- `make win=1 cleanall` delete the `obj/lib/` directories directly
- `make CC="your_compiler_path" CCFLAGS="your_compiling_options" LDFLAGS="your_link_options"` to use custom compiler and options

Result

After compilation completes, `./lib` and `./obj` directories are created (or updated if they already exist), along with a platform-specific subdirectory under each of them.

Examples

Linux example: (The recommended compiler is gcc 4.8.3 or above):

```
make CC="/usr/local/bin/gcc" CCFLAGS="-O3 -I../include -I./  
-D __LINUX__ -D REVERSE_BYTES -m64 -msse2 -mfpmath=sse -fno-omit  
-frame-pointer -fpic -shared " LDFLAGS="-m64 -fpic -shared -lm"
```

Note:

Generally, you only need to use `make CC=<path>` to designate your compiler.
But if you change `CCFLAGS` and `LDFLAGS`, then remember the following:

`-D __LINUX__ -DREVERSE_BYTE` must be specified for Linux.

The following compiler options can be used together or in partial combination to obtain different settings: `DEBUG=1`, `clean` or `CC=`, `CCFLAGS=`, and `LDFLAGS=`.

MOSRA Library Quality

To ensure MOSRA library quality, model developer should run Valgrind, Coverity, or equivalent on the MOSRA model, and clean up all the reported errors before the release.

Setting Up the Runtime Shared-Library Path

When the shared library is ready to use, you must update the shared model search path (defined in the `.OPTION MOSRAPATH` on page 41 option or the `hspice_mosra_models` environment variable) so that the system dynamic loader can find the new MOSRA API shared library.

To update the path, enter:

```
setenv hspice_mosra_models LIB_DIRECTORY/PLATFORM
```

or

```
.option MOSRAPATH = 'LIB_DIRECTORY/PLATFORM'
```

Netlist Setup

To run the PrimeSim MOSRA tool analysis with a customer reliability model, you must add the following option to the input netlist:

```
.OPTION MRAAPI=1
```

Modify the MOSRA model card: use the model level defined in your MOSRA model and assign values to your MOSRA model parameters. See [.OPTION MRAAPI on page 43](#).

For example:

```
.model usrmosra mosra level=102 DDDrela=1e-2 DDDreld=1e-2
```

Other Functions Supported in MOSRA API

This section covers the following topics

- [Accessing Internal Variables of the MOSRA API Models](#)
- [mraPrint\(\) Function](#)

Accessing Internal Variables of the MOSRA API Models

To output internal variables defined in the MOSRA API model, use the following syntax and perform the steps described. Then use the new output format to access these variables.

Syntax

```
.print MOSRA(instance_name, variable_name)
```

Example

In this example, `param1` is one of the internal parameters, and `Mfet1` is the name of the MOSFET (a name with full hierarchical path is also accepted).

```
.print MOSRA(Mfet1, param1)
```

Using the demo MRA model as an example: To print out a variable in the MRAeval.c function, use the following steps to modify the API code:

1. Add a new instance parameter in the structure's MRAinstance (the instance parameter structure, which is defined in MRAdefs.h).

```
typedef struct sMRAinstance
{
    str_T      MRAName;
    MRAstress_t isub;
    MRAstress_t hci;
    MRAstress_t nbti;
    MRAstress_t stress[N_TOTAL_STRESS];;
    /* you can add your instance parameters here */
    Double MRAparam1; /* the new parameter */
}
```

Add this definition to the MRAdefs.h:

```
#define MRA_PARAM1    1
```

2. Store the internal value in here ->MRAparam1:

For example, in MRAeval.c, there is the equation:

```
deltavth = model->MRAREla * pow(inOut->vgs, model->MRARElb);
```

To store the value of `pow(inOut->vgs, model->MRARElb)`,

add the following line to the MRAeval.c file:

```
here-> MRAparam1 = pow(inOut->vgs, model->MRARElb) ;
```

3. Add new codes in MRA.c:

```
MRAparm_t MRAipTable[] = {
    PARAM("xxx",MRA_IGNORED),
    /* new codes for the printback */
    PARAM("param1",MRA_PARAM1),
};
```

4. Add new codes in MRAiask.c:

```
switch (id) {
case MRA_IGNORED:
    *value = 0.0;
    return (MOSRA_OK);

/* new codes for the printback*/
case MRA_param1:
    *value = here->MRAparam1;
    return (MOSRA_OK);
```

```
default:
return (MOSRA_E_BADPARAM);
}
```

In the netlist, you can use the following print output command:

```
.print tran MOSRA(Mfet1, param1)
```

The `param1` should match the variable name in `PARAM("param1",MRA_PARAM1)`, which is defined in the `MRA.c` file.

mraPrint() Function

The MOSRA API provides a printing function: `mraPrint()`. This function can be called from MOSRA API functions to print out information in the `*.mrapi#` file.

The usage of `mraPrint()` is similar to the C function `printf()`.

Note:

This function is supported in the PrimeSim HSPICE simulator only.

MOSRA API Supported Aging Degradations

The section covers the following topics:

- [MOSRA Defined Aging Degradations](#)
- [Degradation Types on Base Model Parameters](#)
- [Other Degradation Types](#)
- [BSIM4 Additional Aging Degradation Types](#)
- [BSIM4 Gate-Leakage Aging Degradation Types](#)
- [TFT \(Level 62 MOSFET\) Model Parameters](#)
- [PSP \(Level 69 MOSFET\) Model Parameters](#)
- [BSIM-CMG \(Level 72 MOSFET\) Model Parameters](#)
- [HiSIM-HV \(Level 73 MOSFET\) Model Parameters](#)
- [UTSOI \(Level 76 MOSFET\) Model Parameters](#)
- [BSIM-BULK \(Level 77 MOSFET\) Model Parameters](#)
- [BSIM-IMG \(Level 78 MOSFET\) Model Parameters](#)
- [IDS Degradation Types for all Models](#)

- [Degradation Types for Instance Parameters](#)
- [Other Degradation Types](#)
- [User-defined Degradation Types](#)

MOSRA Defined Aging Degradations

MOSRA predefines some aging degradations. Each aging degradation applies to one specific compact model parameter or device electrical property. These predefined degradations are defined in data structure `StressType_E` in `MOSRAgendefs.h` file.

Degradation Types on Base Model Parameters

Table 11 Degradation Types On Base Model Parameters

Affected Base Parameters	DEL Degradation (DEL_P)	MUL Degradation (MUL_P)	Binning Degradation (DL_P, DW_P, DP_P)
VTH0 ¹	STRESS_VTH	STRESS_MULVTH0	STRESS_DLVTH0 STRESS_DWVTH0 STRESS_DPVTH0
U0	STRESS_DELU0	STRESS_MU	STRESS_DLU0 STRESS_DWU0 STRESS_DPU0
UA	STRESS_DELUA	STRESS_MULUA	STRESS_DLUA STRESS_DWUA STRESS_DPUA
UB	STRESS_DELUB	STRESS_MULUB	STRESS_DLUB STRESS_DWUB STRESS_DPUB
UC	STRESS_DELUC	STRESS_MULUC	STRESS_DLUC STRESS_DWUC STRESS_DPUC

1. For applying `STRESS_VTH` degradation to `VFB`, in addition to `VTH0` for `BSIM4` and `BSIMSOI` use the `MRAMOD` option. If `MRAMOD=0` default, `stress_VTH` impacts `VTH` model parameters only in post-simulation. If `MRAMOD=1`, `stress_VTH` impacts both `VTH` and `VFB` model parameters in post-simulation.

Table 11 Degradation Types On Base Model Parameters (Continued)

Affected Base Parameters	DEL Degradation (DEL_P)	MUL Degradation (MUL_P)	Binning Degradation (DL_P, DW_P, DP_P)
VSAT	STRESS_DELVSAT	STRESS_MULVSAT	STRESS_DLVSAT STRESS_DWVSAT STRESS_DPVSAT
K1	STRESS_DELK1	STRESS_MULK1	STRESS_DLK1 STRESS_DWK1 STRESS_DPK1
K2	STRESS_DELK2	STRESS_MULK2	STRESS_DLK2 STRESS_DWK2 STRESS_DPK2
A0	STRESS_DELA0	STRESS_MULA0	STRESS_DLA0 STRESS_DWA0 STRESS_DPA0
ETA0	STRESS_DELETA0	STRESS_MULETA0	STRESS_DLETA0 STRESS_DWETA0 STRESS_DPETA0
NFACTOR	STRESS_DELNFACTOR	STRESS_MULNFACTOR	STRESS_DLNFACTOR STRESS_DWNFACTOR STRESS_DPNFACTOR
VOFF	STRESS_DVOFF	STRESS_MULVOFF	STRESS_DLVOFF STRESS_DWVOFF STRESS_DPVOFF
PCLM	STRESS_DPCLM	STRESS_MULPCLM	STRESS_DLPCLM STRESS_DWPCLM STRESS_DPPCLM
			STRESS_DLAGS STRESS_DWAGS

Table 11 Degradation Types On Base Model Parameters (Continued)

Affected Base Parameters	DEL Degradation (DEL_P)	MUL Degradation (MUL_P)	Binning Degradation (DL_P, DW_P, DP_P)
AGS	STRESS_DAGS	STRESS_MULAGS	STRESS_DPAGS
RDSW	STRESS_DRDSW	STRESS_MULRDSW	STRESS_DLRDSW STRESS_DWRDSW STRESS_DPRDSW
CDSCD	STRESS_DELCDSCD	STRESS_MULCDSCD	STRESS_DPCDSCD STRESS_DLCDSCD STRESS_DWCDSCD

Note:

- The effective model parameter is: $Paged = P_{fresh} * (1.0 - MUL_P) + DEL_P + DL_P/Leff + DW_P/Weff + DP_P/Weff/Leff$
- Degradation types in [Table 11](#) supports the BSIM3, BSIM4, BSIM3SOI, BSIMSOI4, and HVMOS models.

Other Degradation Types

Table 12 Other Degradation Types

Affected Parameters	Degradation Type	Operations on BSIM3/BSIM4/BSIMSOI within PrimeSim MOSRA
VTH0	STRESS_DVTLIN	For fresh-simulation only
VTH0	STRESS_DVTLIN_HCI	For fresh-simulation only
VTH0	STRESS_DVTLIN_NBTI	For fresh-simulation only
VTH0	STRESS_DVTSAT	For fresh-simulation only
VTH0	STRESS_DVTSAT_HCI	For fresh-simulation only
VTH0	STRESS_DVTSAT_NBTI	For fresh-simulation only
IDS(channel current)	STRESS_DIDS	$IDS = (1 - STRESS_DIDS) * IDS$
IDS(channel current)	STRESS_DIDSF	When $(vds > 0)$ $IDS = (1 - STRESS_DIDSF) * IDS$
IDS(channel current)	STRESS_DIDSR	When $(vds < 0)$ $IDS = (1 - STRESS_DIDSR) * IDS$

Table 12 Other Degradation Types (Continued)

Affected Parameters	Degradation Type	Operations on BSIM3/BSIM4/BSIMSOI within PrimeSim MOSRA
IDS(channel current)	STRESS_DIDSLIN	For fresh-simulation only
IDS(channel current)	STRESS_DIDSLIN_HCI	For fresh-simulation only
IDS(channel current)	STRESS_DIDSLIN_NBTI	For fresh-simulation only
IDS(channel current)	STRESS_DIDSSAT	For fresh-simulation only
IDS(channel current)	STRESS_DIDSSAT_HCI	For fresh-simulation only
IDS(channel current)	STRESS_DIDSSAT_NBTI	For fresh-simulation only
VTH0	STRESS_DVT_HCI	For fresh-simulation only
VTH0	STRESS_DVT_BTI	For fresh-simulation only
U0	STRESS_MU_HCI	For fresh-simulation only
U0	STRESS_MU_BTI	For fresh-simulation only

Note:

“For fresh-simulation only” indicates that the degradation is shown in the RADEG file but does not affect the post stress simulation.

BSIM4 Additional Aging Degradation Types

Table 13 BSIM4 Additional Aging Degradation Types

Affected Base Parameters	DEL Degradation (DEL_P)	MUL Degradation (MUL_P)	Binning Parameter Degradation
PDIBLC1	STRESS_DELPDIB LC1	STRESS_MULPDIB LC1	STRESS_DLPDIBLC1 STRESS_DWPDIBLC1 STRESS_DPPDIBLC1
PDIBLC2	STRESS_DELPDIB LC2	STRESS_MULPDIB LC2	STRESS_DLPDIBLC2 STRESS_DWPDIBLC2 STRESS_DPPDIBLC2
DELTA	STRESS_DELDELTA	STRESS_MULDELTA	STRESS_DLDELTA STRESS_DWDELTA STRESS_DPDELTA

Table 13 BSIM4 Additional Aging Degradation Types (Continued)

Affected Base Parameters	DEL Degradation (DEL_P)	MUL Degradation (MUL_P)	Binning Parameter Degradation
VOFFCV	STRESS_DELVOFFCV	STRESS_MULVOFFCV	STRESS_DLVOFFCV STRESS_DWVOFFCV STRESS_DPVOFFCV
CGDO	STRESS_DELCGDO	STRESS_MULCGDO	none
CGSO	STRESS_DELCGSO	STRESS_MULCGSO	none
CGDL	STRESS_DELCGDL	STRESS_MULCGDL	none
CGSL	STRESS_DELCGSL	STRESS_MULCGSL	none
NOFF	STRESS_DELNOFF	STRESS_MULNOFF	none
DLC	STRESS_DELDLC	none	none
NOIA	none	STRESS_MULNOIA	none
NOIB	none	STRESS_MULNOIB	none
NOIC	none	STRESS_MULNOIC	none
EM	none	STRESS_MULEM	none
AF	none	STRESS_MULAF	none
KF	none	STRESS_MULKF	none

BSIM4 Gate-Leakage Aging Degradation Types

Table 14 BSIM4 Gate-Leakage Aging Degradation Types

Affected Parameters	DEL degradation	MUL degradation	Binning Degradation (DL_P, DW_P, DP_P)
MINV	none	STRESS_MULMINV	STRESS_MLMINV
			STRESS_MWMINV
			STRESS_MPMINV
TOXREF	none	STRESS_MULTOXREF	none
			none

Table 14 BSIM4 Gate-Leakage Aging Degradation Types (Continued)

Affected Parameters	DEL degradation	MUL degradation	Binning Degradation (DL_P, DW_P, DP_P)
			none
POXEDGE	none	STRESS_MULPOXEDGE	STRESS_MLPOXEDGE
			STRESS_MWPOXEDGE
			STRESS_MPPOXEDGE
NTOX	none	STRESS_MULNTOX	STRESS_MLNTOX
			STRESS_MWNTOX
			STRESS_MPNTOX
DLCIG	none	STRESS_MULDLCIG	none
			none
			none
DLCIGD	none	STRESS_MULDLCIGD	none
			none
			none
AIGC	none	STRESS_MULAIGC	STRESS_MLAIGC
			STRESS_MWAIGC
			STRESS_MPAIGC
BIGC	none	STRESS_MULBIGC	STRESS_MLBIGC
			STRESS_MWBIGC
			STRESS_MPBIGC
CIGC	none	STRESS_MULCIGC	STRESS_MLCIGC
			STRESS_MWCIGC
			STRESS_MPCIGC
NIGC	none	STRESS_MULNIGC	STRESS_MLNIGC
			STRESS_MWNIGC
			STRESS_MPNIGC

Table 14 BSIM4 Gate-Leakage Aging Degradation Types (Continued)

Affected Parameters	DEL degradation	MUL degradation	Binning Degradation (DL_P, DW_P, DP_P)
PIGCD	none	STRESS_MULPIGCD	STRESS_MLPIGCD
			STRESS_MWPIGCD
			STRESS_MPPIGCD
AIGSD	none	STRESS_MULAIGSD	STRESS_MLAIGSD
			STRESS_MWAIGSD
			STRESS_MPAIGSD
BIGSD	none	STRESS_MULBIGSD	STRESS_MLBIGSD
			STRESS_MWBIGSD
			STRESS_MPBIGSD
CIGSD	none	STRESS_MULCIGSD	STRESS_MLCIGSD
			STRESS_MWCIGSD
			STRESS_MPCIGSD
VFBSDOFF	STRESS_DELVFBSDOFF	none	STRESS_DLVFBSDOFF
			STRESS_DWVFBSDOFF
			STRESS_DPVFBSDOFF
AIGS	none	STRESS_MULAIGS	STRESS_MLAIGS
			STRESS_MWAIGS
			STRESS_MPAIGS
BIGS	none	STRESS_MULBIGS	STRESS_MLBIGS
			STRESS_MWBIGS
			STRESS_MPBIGS
CIGS	none	STRESS_MULCIGS	STRESS_MLCIGS
			STRESS_MWCIGS
			STRESS_MPCIGS
AIGD	none	STRESS_MULAIGD	STRESS_MLAIGD

Table 14 BSIM4 Gate-Leakage Aging Degradation Types (Continued)

Affected Parameters	DEL degradation	MUL degradation	Binning Degradation (DL_P, DW_P, DP_P)
			STRESS_MWAIGD
			STRESS_MPAIGD
BIGD	none	STRESS_MULBIGD	STRESS_MLBIGD
			STRESS_MWBIGD
			STRESS_MPBIGD
CIGD	none	STRESS_MULCIGD	STRESS_MLCIGD
			STRESS_MWCIGD
			STRESS_MPCIGD
NFACTOR	STRESS_DELNFACTOR	STRESS_MULNFACTOR	STRESS_MLNFACTOR
			STRESS_MWNFACTOR
			STRESS_MPNFACTOR
VOFF	STRESS_DVOFF	STRESS_MULVOFF	STRESS_MLVOFF
			STRESS_MWVOFF
			STRESS_MPVOFF

MUL degradation equation:

$$\text{MINV}(\text{aged}) = \text{MINV} * (1 - \text{STRESS_MULMINV})$$

$$\text{LMINV}(\text{aged}) = \text{LMINV} * (1 - \text{STRESS_MLMINV})$$

$$\text{WMINV}(\text{aged}) = \text{WMINV} * (1 - \text{STRESS_MWMINV})$$

$$\text{PMINV}(\text{aged}) = \text{PMINV} * (1 - \text{STRESS_MPMINV})$$

DEL degradation equation:

$$\text{VFBSDOFF}(\text{aged}) = \text{VFBSDOFF} + \text{STRESS_DELVFBSDOFF}$$

$$\text{LVFBSDOFF}(\text{aged}) = \text{LVFBSDOFF} + \text{STRESS_DLVFBSDOFF}$$

$$\text{WVFBSDOFF}(\text{aged}) = \text{WVFBSDOFF} + \text{STRESS_DWVFBSDOFF}$$

$$\text{PVFBSDOFF}(\text{aged}) = \text{PVFBSDOFF} + \text{STRESS_DPVFBSDOFF}$$

TFT (Level 62 MOSFET) Model Parameters

Table 15 Degradation Types On TFT (Level 62 MOSFET) Model Parameters

Affected Model Parameters	DEL Degradation	MUL Degradation
VTO	STRESS_VTH	STRESS_MULVTH0
MU0	STRESS_DELUA	STRESS_MULUA
MU1	STRESS_DELU0	STRESS_MULU0
ETA	STRESS_DELNFACTOR	STRESS_MULNFACTOR
MUS	STRESS_DELUC	STRESS_MULUC
LAMBDA	STRESS_DPCLM	STRESS_MULPCLM
BLK	STRESS_DELETA0	STRESS_MULETA0
VOFFCV	STRESS_DVOFF	STRESS_MULVOFF

PSP (Level 69 MOSFET) Model Parameters

Table 16 Degradation Types On PSP (level 69 MOSFET) Model Parameters

Affected Model Parameters	DEL Degradation	MUL Degradation
VFBO	STRESS_DELVTH	STRESS_MULVTH
UO	STRESS_DELU0	STRESS_MULU0
MUEO	STRESS_DELUA	STRESS_MULUA
THESATO	STRESS_DELVSAT	STRESS_MULVSAT
THESATL	STRESS_DELVSATL	STRESS_MULVSATL
CFL	STRESS_DELETA0	STRESS_MULETA0
CTO	STRESS_DELNFACTOR	STRESS_MULNFACTOR
ALPL	STRESS_DELPCLM	STRESS_MULPCLM
ALP1L1	STRESS_DELALP1	STRESS_MULALP1
ALP2L1	STRESS_DELALP2	STRESS_MULALP2
RSW1	STRESS_DELRDSW	STRESS_MULRDSW

BSIM-CMG (Level 72 MOSFET) Model Parameters

Table 17 Degradation Types On BSIM-CMG (Level 72 MOSFET) Model Parameters

Affected Parameters	DEL Degradation	MUL Degradation
DELVTRAND	STRESS_VTH	none
U0	STRESS_DELU0	STRESS_MU
UA	STRESS_DELUA	STRESS_MULUA
UC	STRESS_DELUC	STRESS_MULUC
ETA0	STRESS_DELETA0	STRESS_MULETA0
VSAT	STRESS_DELVSAT	STRESS_MULVSAT
Effective U0	None	STRESS_MEU0

HiSIM-HV (Level 73 MOSFET) Model Parameters

Table 18 Degradation Types On HiSIM-HV (Level 73 MOSFET) Model Parameters

Affected Model Parameters	DEL Degradation	MUL Degradation (MUL_P)
VFBC	STRESS_DELVFBC	none
MUECB0	none	STRESS_MULMUECB0
MUECB1	none	STRESS_MULMUECB1
MUEPH0	none	STRESS_MULMUEPH0
MUEPH1	none	STRESS_MULMUEPH1
MUESR0	none	STRESS_MULMUESR0
MUESR1	none	STRESS_MULMUESR1
RD	STRESS_DELRD	STRESS_MULRD
RDVD	STRESS_DELRDVD	STRESS_MULRDVD
VMAX	none	STRESS_MULVMAX
RTH0	STRESS_DELRTH0	STRESS_MULRTH0

UTSOI (Level 76 MOSFET) Model Parameters

Table 19 Degradation Types On UTSOI (Level 76 MOSFET) Model Parameters

Affected Model Parameters	DEL Degradation	MUL Degradation
VFBO	STRESS_DELVTH	STRESS_MULVTH
UO	STRESS_DELU0	STRESS_MULU0
MUEO	STRESS_DELUA	STRESS_MULUA
THESATO	STRESS_DELVSAT	STRESS_MULVSAT
THESATL	STRESS_DELVSATL	STRESS_MULVSATL
THESATBO	STRESS_DELVSATB	STRESS_MULVSATB
CFL	STRESS_DELETA0	STRESS_MULETA0
CTO	STRESS_DELNFACTOR	STRESS_MULNFACTOR
ALPL	STRESS_DELPCLM	STRESS_MULPCLM
ALP1L1	STRESS_DELALP1	STRESS_MULALP1
RSW1	STRESS_DELRDSW	STRESS_MULRDSW

BSIM-BULK (Level 77 MOSFET) Model Parameters

Table 20 Degradation Types On BSIM-BULK (Level 77 MOSFET) Model Parameters

Affected Model Parameters	DEL Degradation	MUL Degradation (MUL_P)
VFB	STRESS_VTH	STRESS_MULVTH
U0	STRESS_DELU0	STRESS_MULU0
UA	STRESS_DELUA	STRESS_MULUA
UC	STRESS_DELUC	STRESS_MULUC
VSAT	STRESS_DELVSAT	STRESS_MULVSAT
K1	STRESS_DELK1	STRESS_MULK1
K2	STRESS_DELK2	STRESS_MULK2

Table 20 *Degradation Types On BSIM-BULK (Level 77 MOSFET) Model Parameters (Continued)*

Affected Model Parameters	DEL Degradation	MUL Degradation (MUL_P)
ETA0	STRESS_DELETA0	STRESS_MULETA0
NFACTOR	STRESS_DELNFACTOR	STRESS_MULNFACTOR
PCLM	STRESS_DELPCLM	STRESS_MULPCLM
RDSW	STRESS_DELRDSW	STRESS_MULRDSW
CDSCD	STRESS_DELCDSCD	STRESS_MULCDSCD

BSIM-IMG (Level 78 MOSFET) Model Parameters

Table 21 *Degradation Types On BSIM-IMG (level 78 MOSFET) Model Parameters*

Affected Parameters	DEL Degradation	MUL Degradation
Threshold voltage	STRESS_DELVTH	STRESS_MULVTH

IDS Degradation Types for all Models

Table 22 *IDS Degradation Types For All Models*

Affected Parameters	DEL Degradation	MUL Degradation
IDS (channel current)	STRESS_DIDS	$IDS = (1 - STRESS_DIDS) * IDS$
IDS (channel current)	STRESS_DIDSF	When $(v_{ds} > 0)$ $IDS = (1 - STRESS_DIDSF) * IDS$
IDS (channel current)	STRESS_DIDSR	When $(v_{ds} < 0)$ $IDS = (1 - STRESS_DIDSR) * IDS$

Degradation Types for Instance Parameters

Table 23 Degradation Types For MOSFET Model Instance Parameters

Affected Model Parameters	DEL Degradation	MUL Degradation
DTEMP	STRESS_DTEMP	None

Other Degradation Types

Table 24 Other Supported Degradation Types

Degradation Type	Operations
STRESS_DUMMY	For fresh-simulation only

User-defined Degradation Types

The PrimeSim MOSRA tool introduces the mechanism to allow your aging model to modify any compact model parameter that the custom aging model requires. In this flow, predefined MOSRA aging degradations in MOSRAgends.h are ignored. User-defined degradations to compact model parameters are provided to the simulator through the MOSRA API interface function.

Examples

This example shows how to define your own degradations and their impact to compact model parameters in a custom MOSRA model.

1. Define degradation types in the MRAdefs.h file:

```
/* degradation type */
#define STRESS_VTH0          0
#define STRESS_VSAT          1
```

Note:

The value of degradation type cannot be larger than STRESS_TOTALSIZE (this number is defined in MOSRAgends.h)

2. Define stress information at the header of MRASetsMod.c file:

```
static MRAsstressInfo_t stressInfo[]={ {STRESS_VTH0 , "vth0" , 0},
{STRESS_VSAT, "vsat", 0},
};
```

MRAsstressInfo data structure is defined in MOSRAgends.h:

```
typedef struct sMRastressInfo {
    INT32_t      stress_type;    /* user-defined degradation type */
    const char   *param_name;    /* related compact model parameter that
aging model will change */
    INT32_t      stress_mode;    /* now only 0 (aged_mode) supported */
} MRastressInfo_t;
```

Note:

`stress_mode = 0 (AGED_MODE)` means the MOSRA model returns aging model parameter ($P_{\text{Fresh}} + \Delta P_{\text{age}}$) value to the simulator. This requires the aging model to read in a fresh model parameter.

In this example, the aging model does the degradation for model parameter `vth0` and `vsat`.

3. Send stress information to the simulator:

At the end of the `MRAsSetupModel` function, add the following statement:

```
ckt->stressInfo = stressInfo;
ckt->stressInfoSize = sizeof(stressInfo)/sizeof(MRastressInfo_t);
```

Note:

- User-defined stress type feature is supported in PrimeSim HSPICE, PrimeSim SPICE, PrimeSim Pro and PrimeSim XA.
- User-defined stress type feature supports: BSIM4, PSP, BSIMCMG, HiSIMHV, HiSIM2, BSIMBULK, UTSOI and BSIMIMG.

The simulator writes aging model parameter value into the `.radeg` file and these aged model parameters are used in post stress simulation.

See example code in MOSRA example package under `mosraapi/MRAFMF`.

MOSRA API Data Structures

```
typedef struct sMRA_ENV {
    REAL64_t      CKTtemp;
    REAL64_t      CKTtnom;
    unsigned      CKTtempGiven:1;
    unsigned      CKTtnomGiven:1;
    MRA_OPT_PATH  *MRAoptPath;
    char          *MRAoutput;
    /* output file name without postfix that follows simulator's convention.
User can define the Postfix
as needed and append together with alterCount in API for final output
name */
    INT32_t      alterCount;          /* count of .alter statement */
    REAL64_t      relTimePoint;       /* reltime point */
}
```

```

    MRAinstGen      *instSrc;                /* pointer to store "source"
instance */
    unsigned        hci:1;                  /* .mosra hci = 0 | 1 */
    unsigned        bti:1;                  /* .mosra bti = 0 | 1 */
    unsigned        tddb:1;                 /* .mosra tddb = 0 | 1 */
    INT32_t          circuit_report;         /* .mosra circuit_report =
value, user defined circuit report value */
    REAL64_t         trelax;                 /* .mosra trelax = value, user
defined relaxation time */
    REAL64_t         area_scaling;          /* .mosra area_scaling = value,
user defined area scaling */
    REAL64_t         relTotalTime;          /* .mosra relTotalTime = value
*/
    REAL64_t         ageThreshold;          /* .mosra ageThreshold = value,
*/
    INT32_t          relTimeIdx;            /* index of the current reltime
*/
    unsigned         earlyExitEnabled:1;    /* .mosra earlyExitEnabled =
value */
    MRAstressInfo_t *stressInfo; /* user defined stress information */
    INT32_t stressInfoSize; /* user defined stress information size */
    INT32_t SimMode; /* .mosra simmode */
    char *relStepCmd; /* .mosra reltimepoints/dec/lin/log/relstep */
} MRA_ENV;

```

MOSRA API Model Function Prototypes

Table 25 Model function prototypes

Function Prototype	Description
int MRASetModParam(mod, id, value) Returns: MOSRA function return status defined in MOSRAgendef.h Arguments: <i>mod</i> input (struct MRAmodGen *): Pointer to the MRA model parameter structure. <i>id</i> input (int): Model parameter id. <i>value</i> input (double): Model parameter value which is sent back to simulator	This function sets up MRA model parameter value according to its id.
int MRAgetModParam(mod, id, value) Returns: MOSRA function return status defined in MOSRAgendef.h Arguments: <i>mod</i> input (struct MRAmodGen *): Pointer to the MRA model parameter structure. <i>id</i> input (int): Model parameter id. <i>value</i> output (double *): Model parameter value which is sent back to simulator	This function provides the interface for the simulator to query the MRA model parameter according to the parameter id.
int MRAgetInstParam(inst, id, value) Returns: MOSRA function return status defined in MOSRAgendef.h Arguments: <i>inst</i> input (struct MRAinstGen *): Pointer to the MRA instance parameter structure <i>id</i> input (int): Instance parameter id <i>value</i> output (double *): Instance parameter value which is sent back to simulator.	This function provides the interface for the simulator to query the MRA instance parameter value according to the parameter id.

Table 25 Model function prototypes (Continued)

Function Prototype	Description
int MRASETInstParam(inst, id, value) Returns: MOSRA function return status defined in MOSRAgendef.h Arguments: - inst input (struct MRAinstGen *): Pointer to the MRA instance parameter structure. - id input (int): Instance parameter id. - value input (double): Instance parameter value.	This function sets up MRA instance parameter value according to its id.
int MRASETupModel(mod, ckt) Returns: MOSRA function return status defined in MOSRAgendef.h Arguments: - mod input (struct MRAmodGen *): Pointer to the MRA model parameter structure - ckt input (struct MRA_ENV *): Pointer to the circuit environment parameter structure	This function sets MRA model parameter default value.
int MRASETupInst(inst, ckt) Returns: MOSRA function return status defined in MOSRAgendef.h Arguments: - inst input (struct MRAinstGen *): Pointer to the MRA instance parameter structure - ckt input (struct MRA_ENV *): Pointer to the circuit environment parameter structure	This function sets MRA instance parameter default value.
int MRAtemp(mod, inst, ckt) Returns: MOSRA function return status defined in MOSRAgendef.h Arguments: - mod input (struct MRAmodGen *): Pointer to the MRA model parameter structure - inst input (struct MRAinstGen *): Pointer to the MRA instance parameter structure - ckt input (struct MRA_ENV *): Pointer to the circuit environment parameter structure	This function performs MRA parameter temperature and geometry scaling.
int MRAevaluate(mod, inst, ckt, inOut) Returns: MOSRA function return status defined in MOSRAgendef.h Arguments: - mod input (struct MRAmodGen *): Pointer to the MRA model parameter structure - inst input (struct MRAinstGen *): Pointer to the MRA instance parameter structure - ckt input (struct MRA_ENV *): Pointer to the circuit environment parameter structure - inOut input (struct MRA_VAR *): Pointer to the input/output data structure	This function performs MRA stress equation calculation.

Table 25 Model function prototypes (Continued)

Function Prototype	Description
int MRAinteg(mod, inst, ckt, delta_t, tran_time, stress_time, action, stress_total_lin, stress_total, inOut) Returns: MOSRA function return status defined in MOSRAgendef.h Arguments: • mod input (struct MRAmodGen *): Pointer to the MRA model parameter structure • inst input (struct MRAinstGen *): Pointer to the MRA instance parameter structure • ckt input (struct MRA_ENV *): Pointer to the circuit environment parameter structure • delta_t input (double): Transient simulation time step • tran_time input (double): Total transient simulation (integration) time when .MOSRA agingstart/agingstop command is used, tran_time = agingstop - agingstart • stress_time input (double): Total circuit operation (stress) time • action input (enum MOSRAintegAction_t): Mode control flag • inOut input (struct MRA_VAR *): Pointer to the input/output data structure • stress_total_lin input (struct MRAstress_t *): When action=ACTION_FINAL: Pointer to the structure of total MOSRA stress (Linearized) • stress_total input (struct MRAstress_t *): When action=ACTION_INTEG: Pointer to the structure of total MOSRA stress (Linearized) When action=ACTION_FINAL: Pointer to the structure of total MOSRA stress after extrapolation	This function performs MRA stress integration and extrapolation operation. This function works in two different modes: • When action=ACTION_INTEG, it conducts the integration of stresses. • When action=ACTION_FINAL, it conducts the extrapolation from transient simulation time to circuit operation time.
int MRAlife(mod, inst, prev_deg, delta_t, tran_time, stress_time, degf, deg_name, inOut) Returns: MOSRA function return status defined in MOSRAgendef.h Arguments: • mod input (struct MRAmodGen *): Pointer to the MRA model parameter structure • inst input (struct MRAinstGen *): Pointer to the MRA instance parameter structure • delta_t input (double): Transient simulation time step • tran_time input (double): Total transient simulation (integration) time when .MOSRA agingstart/agingstop command is used, tran_time = agingstop - agingstart • stress_time input (double): Total circuit operation (stress) time • inOut input (struct MRA_VAR *): Pointer to the input/output data structure • degf input (double *): Stress value when device gets the end of life • deg_name input (char *): Name keyword of the stress to do the lifetime calculation	This function performs MRA lifetime computation. The lifetime is sent back to the simulator through the input/output data structure (MRA_VAR).

Table 25 Model function prototypes (Continued)

Function Prototype	Description
int MRAPaged(mod, inst, ckt, stress_value, paged_value) Returns: MOSRA function return status defined in MOSRAgendef.h Arguments: mod input (struct MRAModGen *): Pointer to the MRA model parameter structure - inst input (struct MRAinstGen *): Pointer to the MRA instance parameter structure - ckt input (struct MRA_ENV *): Pointer to the circuit environment parameter structure - stress_value input (MRAstress_t *): The stress value which will affect the model parameters - paged_value output (MRAstress_t *): The model parameters which have been degraded	This function performs MOSFET model parameter degradation calculation based on the amount of stress each device received. This function is invoked when options <i>mraext</i> and <i>mrapped</i> are set.
int MRApostCompute(mod, inst, ckt, inOut, Stress_value, DegPost) Arguments: - mod input (struct MRAModGen *): Pointer to the MRA model parameter structure - inst input (struct MRAinstGen *): Pointer to the MRA instance parameter structure - ckt input (struct MRA_ENV *): Pointer to the circuit environment parameter structure - InOut input (struct MRA_VAR *): Pointer to the input/output data structure - Stress_value input (MRAstress_t *): The stress value which affects the lds/gm/gds/gmbs - DegPost output (REAL64_t *): The lds/gm/gds/gmbs that have been degraded	This function is accessed during the post-stress analysis phase by either .OPTION MRAEXT=1 or OPTION MRA_POST_COMPUTE=1. When .OPTION MRAEXT=1 is set, it computes device quantity degradation on lds, gm, gds, and gmbs based on the amount of stress each device received from stress simulation. When .OPTION MRA_POST_COMPUTE=1 is set, it allows users to scale lds degradation (in percentage) based on the bias condition of the device. You can customize the scaling function as needed. .OPTION MRAEXT=1 has higher precedence than .OPTION MRA_POST_COMPUTE=1.